OSI Model (Open Systems Interconnection Model)

The **OSI Model (Open Systems Interconnection Model)** is a **7-layer conceptual framework** used to understand how communication happens in a computer network. It does not represent a real implementation but serves as a structured way to visualize and reason about networking.

Without such a model, networking can feel like a collection of unrelated protocols. The OSI model provides a clear structure by breaking communication into well-defined layers, each responsible for a specific part of the process.

Why the OSI Model Exists

The OSI model is important because it helps us:

Understand networking: It gives a clear picture of how data moves from one system to another.

Organize responsibilities: Each layer handles a specific function instead of mixing everything together.

Debug effectively: Problems can be isolated to a particular layer.

Explain systems clearly: It helps identify which device or protocol is responsible for what.

Mental Model

Think of networking as a **stack of layers**, where each layer performs a specific task and passes data to the next layer. Each layer only focuses on its own responsibility and does not need to know the internal workings of other layers. This layered approach makes the system modular, easier to understand, and easier to maintain.

What is a Protocol?

A **protocol** is simply a set of communication rules between systems. These rules define:

How communication starts

What format the data should be in

Where the data should go

How the receiver will interpret the data

What happens if something goes wrong

Protocols are the building blocks of networking. Each layer in the OSI model uses specific protocols to perform its function. Examples include:

HTTP: Handles web communication by defining how clients (browsers) request resources and how servers respond with web pages, APIs, and other content.

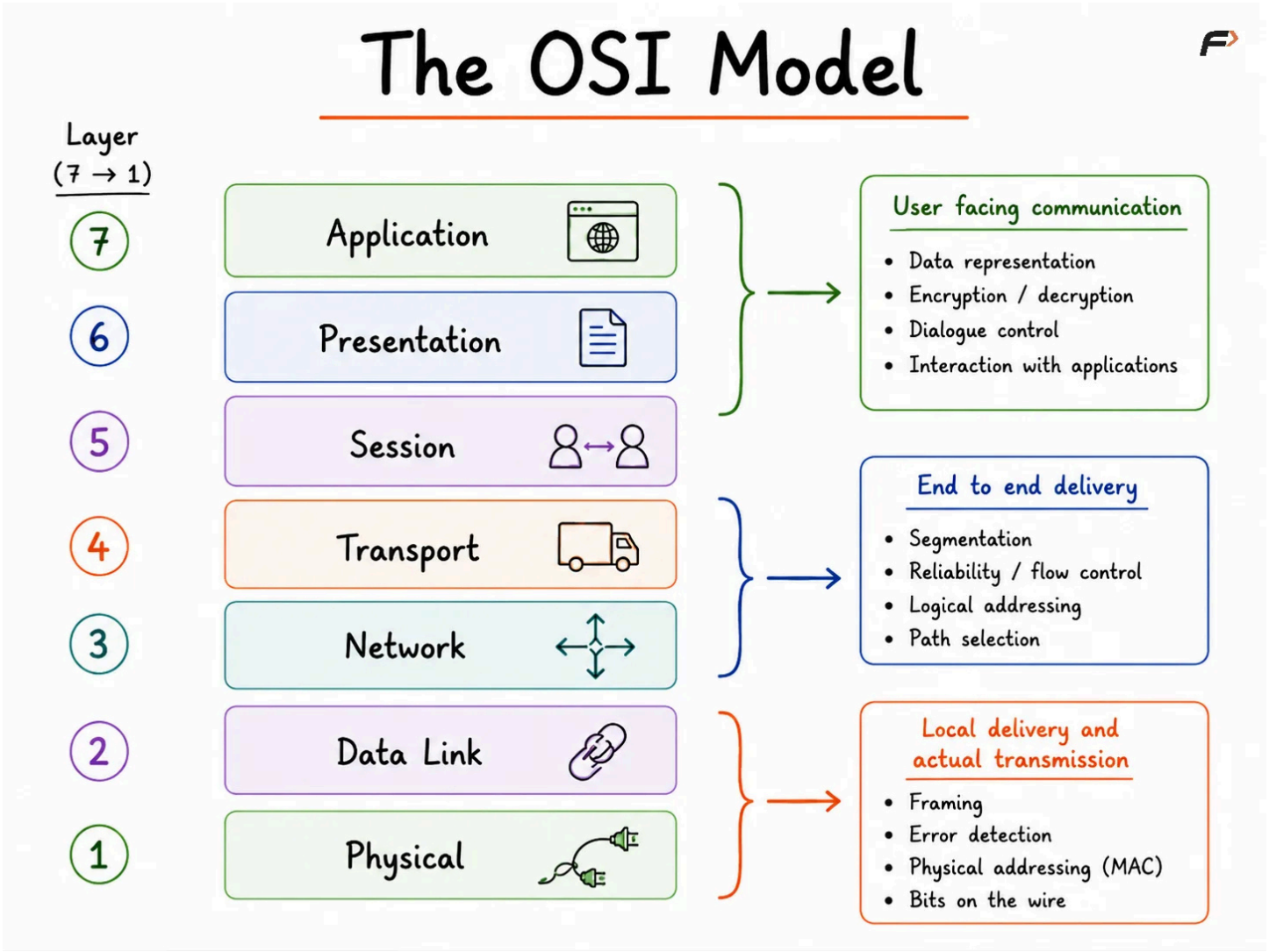
TCP: Ensures reliable data delivery by establishing connections, maintaining packet order, handling retransmissions, and managing flow control between sender and receiver.

IP: Responsible for addressing and routing by assigning unique IP addresses to devices and determining the path data takes across networks.

DNS: Translates human-readable domain names (like google.com) into IP addresses so that devices can locate and communicate with each other.

The OSI Model Layers

The OSI model consists of **7 layers**, each responsible for a specific part of communication:



Each layer builds upon the one below it, forming a complete communication pipeline from application-level interaction down to physical signal transmission.

Who Introduced OSI?

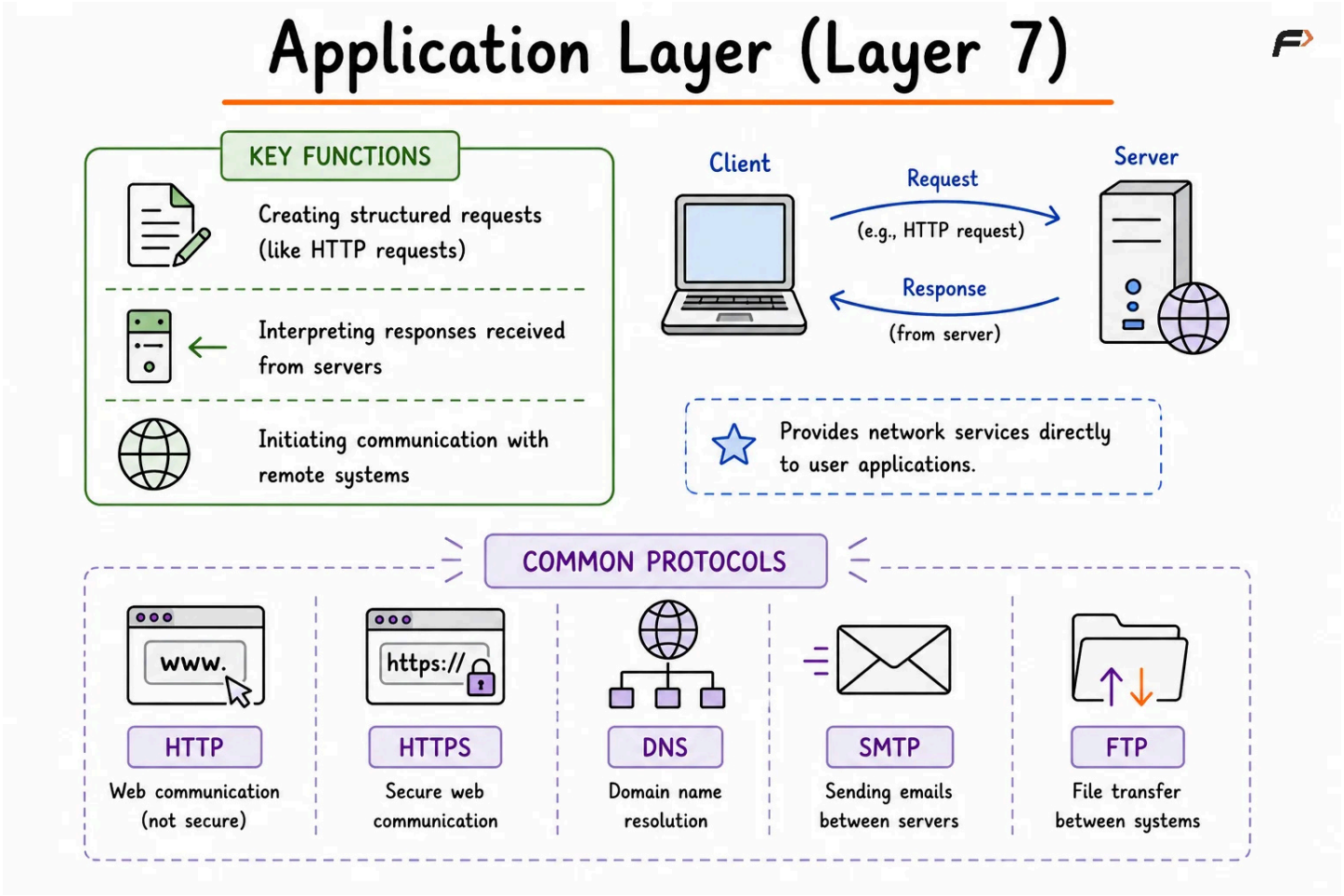
The OSI model was introduced by the **ISO (International Organization for Standardization)** as a reference model. Its goal was to standardize how network communication should be structured so that systems from different vendors and organizations could interoperate seamlessly.

Although modern networking primarily follows the TCP/IP model, the OSI model remains extremely valuable as a **learning, teaching, and debugging tool**.

Application Layer (Layer 7)

The **Application Layer** is the layer closest to the software that the user interacts with. It is important to understand that this does **not mean the entire application UI** is the application layer. Instead, it refers to the part of the application that **uses network communication**.

In simple terms, this is the layer where communication becomes **meaningful to the software**. It is where applications create requests, interpret responses, and interact with network services.



What Happens at the Application Layer

At this layer, the application is responsible for generating data that can be sent over the network. This includes:

Creating structured requests (like HTTP requests)

Interpreting responses received from servers

Initiating communication with remote systems

This is the starting point of network communication from the application's perspective.

Examples

Browser making an HTTP request to load a webpage

Email application sending an email

DNS request to resolve a domain name

File transfer between systems

In all these cases, the application layer defines **what is being communicated**, not how it is physically transmitted.

Common Protocols in the Application Layer

HTTP (HyperText Transfer Protocol): Used to transfer web pages and API data between client and server, defining how requests (GET, POST, etc.) are made and how responses are structured and delivered.

HTTPS (HyperText Transfer Protocol Secure): An extension of HTTP that uses TLS encryption to secure data in transit, ensuring confidentiality, integrity, and protection against eavesdropping or tampering.

DNS (Domain Name System): Converts human-readable domain names (like example.com) into IP addresses, enabling devices to locate and communicate with servers on the internet.

SMTP (Simple Mail Transfer Protocol): Used for sending emails between mail servers and from clients to servers, handling the outgoing mail delivery process.

FTP (File Transfer Protocol): Used to transfer files between systems over a network, supporting operations like upload, download, and directory management (often replaced by more secure alternatives like SFTP).

SSH (Secure Shell): Provides secure remote login and command-line access to systems, using encryption for authentication and communication, commonly used for server administration.

DHCP (Dynamic Host Configuration Protocol): Automatically assigns IP addresses, subnet masks, gateways, and DNS settings to devices, enabling seamless network configuration without manual setup.

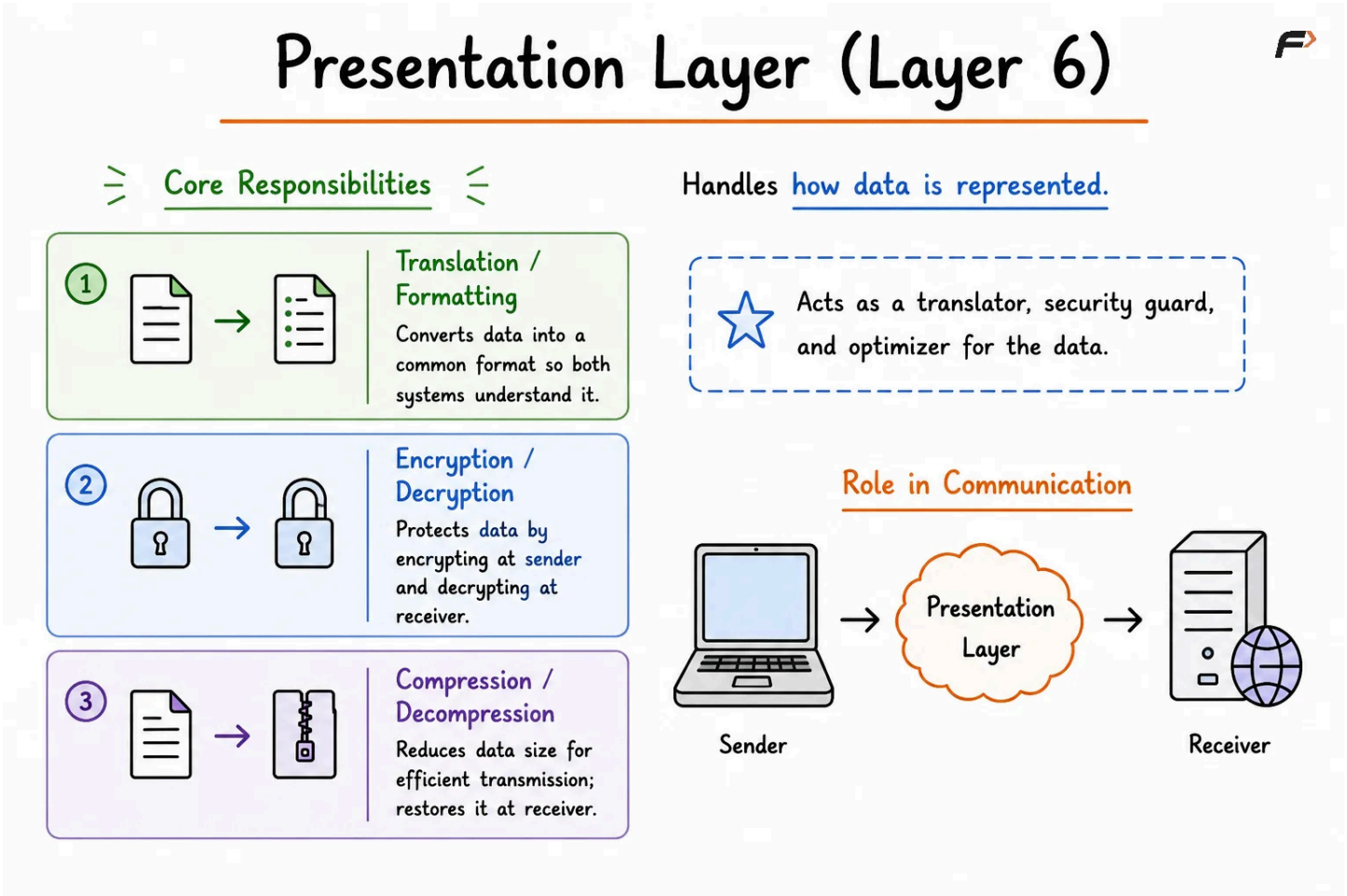
Summary

The Application Layer does not deal with reliability, routing, or physical transmission. It focuses only on **what data is being communicated and how it should be understood by applications**. This is the layer where communication becomes **meaningful to the software**.

Presentation Layer (Layer 6)

The **Presentation Layer** is concerned with **how data is represented**. It ensures that the sender and receiver interpret the data correctly, even if they use different systems or formats.

In simple terms, this layer acts as a **translator and formatter** between systems. It makes sure that the data sent by one system can be properly understood by another.



Core Responsibilities

Translation / Formatting: Converts data into a standardized format so that different systems and applications can interpret it correctly.

Encryption / Decryption: Protects data by encrypting it before transmission and decrypting it at the receiver's end, ensuring confidentiality and secure communication.

Compression / Decompression: Optimizes data transfer by reducing its size before sending and restoring it to its original form at the receiver.

Why This Layer is Needed

When two systems communicate, they must agree on aspects such as data encoding (e.g., UTF-8, ASCII), file formats (e.g., JSON, XML), and encryption methods. Without this, the receiver may fail to correctly interpret the data sent by the sender. For example, text encoded in one format can appear as unreadable characters if decoded incorrectly.

We can say that the **Application Layer** decides **what to send**, while the **Presentation Layer** decides **in what format to send**. This separation ensures that applications can focus on logic, while the presentation layer ensures compatibility and correct interpretation.

Important Note: In real-world systems, the Presentation Layer is **not always implemented as a separate layer**. Its responsibilities are often handled inside applications or libraries. For example:

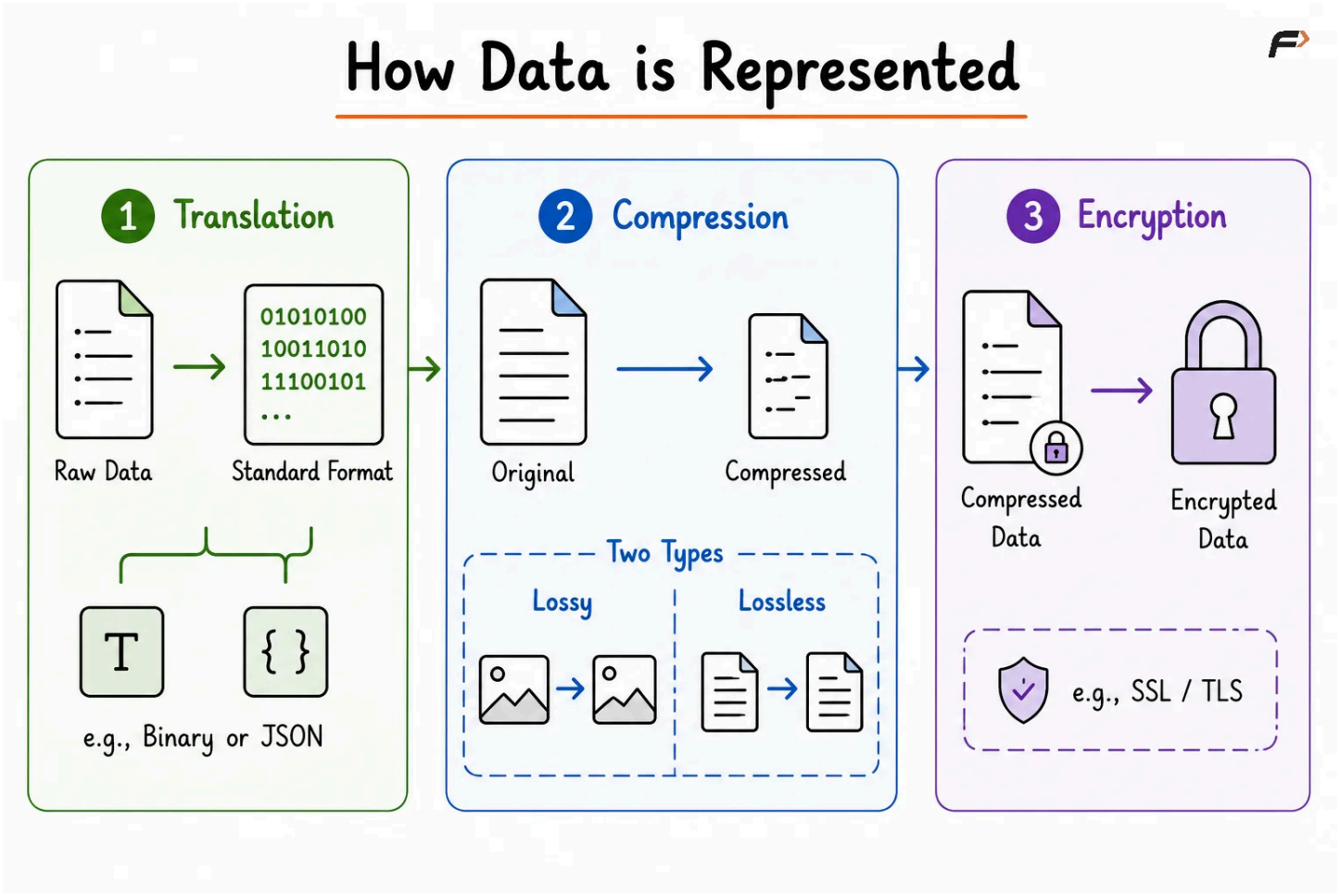
JSON serialization happens inside application code.

TLS encryption is handled by libraries like OpenSSL.

Compression is handled by frameworks or middleware.

Flow of Data

The transformation of data at the Presentation Layer can be understood as a step-by-step process:



Step 1: Translation Raw data is converted into a standard format. Example: Text to Binary (bits) or structured format like JSON.

Step 2: Compression The data is compressed to reduce size for faster transmission. There are two types of compression:

Lossy Compression: Reduces file size by discarding some data, typically used for media like images, audio, and videos where slight quality loss is acceptable.

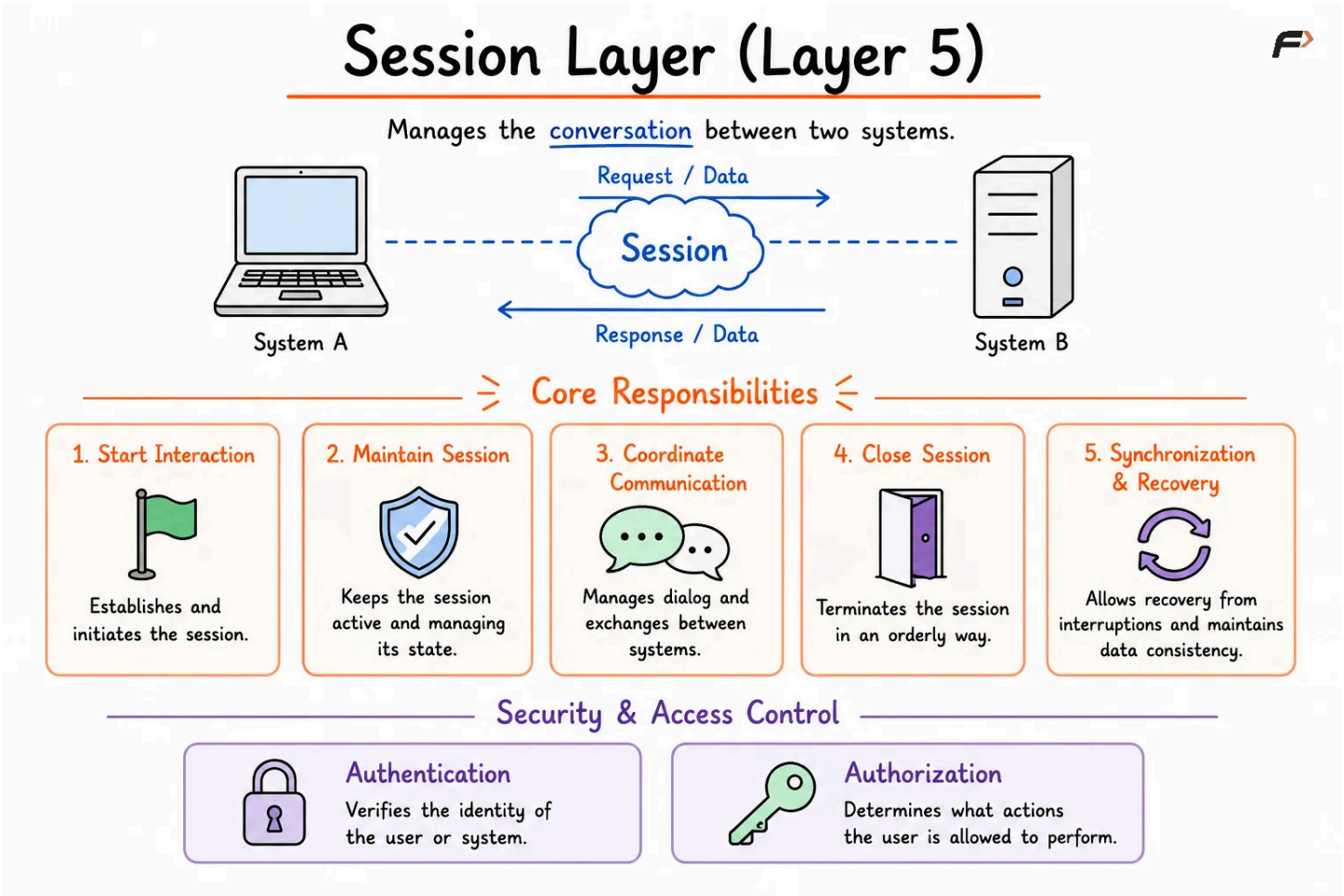
Lossless Compression: Reduces file size without losing any data, ensuring the original content can be perfectly reconstructed, commonly used for text and critical files.

Step 3: Encryption The data is encrypted to ensure security during transmission. Example: SSL/TLS (Secure Sockets Layer / Transport Layer Security).

Session Layer (Layer 5)

The **Session Layer** is responsible for **managing the conversation between two systems**. Its main job is to **establish, maintain, and close sessions**.

It is important to understand that this layer handles the **communication session**, not the raw transport of bits or packets. That responsibility belongs to lower layers like Transport and Network.



Core Responsibility

The Session Layer ensures that communication between two systems is organized as a continuous, meaningful interaction rather than a collection of independent messages.

Starting the interaction: Establishes a session between two systems before data exchange begins.

Maintaining the session: Keeps track that ongoing communication belongs to the same session.

Coordinating communication: Manages multiple back-and-forth messages between systems in an organized way.

Handling session closure: Gracefully terminates the session once communication is complete.

Synchronization and recovery: Helps resume or synchronize communication if it gets interrupted.

In modern systems, session management is typically handled at higher levels such as application frameworks or protocols like HTTP using cookies and tokens. In practice, the Session Layer is not implemented as a distinct layer, as its responsibilities are absorbed by

application-level logic, libraries, frameworks, and protocols, similar to how the Presentation Layer's functions are integrated into real-world implementations.

Examples of Session Management

APIs: Maintaining a session between client and server across multiple requests.

NetBIOS: Used for session-level communication in older systems.

Authentication and Authorization

Session-based systems often involve:

Authentication: Verifying the identity of the user or system.

Authorization: Determining what actions the user is allowed to perform.

While not purely a session-layer function in strict OSI terms, these concepts are closely related to how sessions are maintained in real-world systems.

Real-World Intuition

When you download an image or a video, the data is transmitted in multiple packets from the server to your system. The Session Layer helps maintain the context of this communication ensuring that all packets belong to the same ongoing interaction. It keeps track of:

Which packets are part of the current session

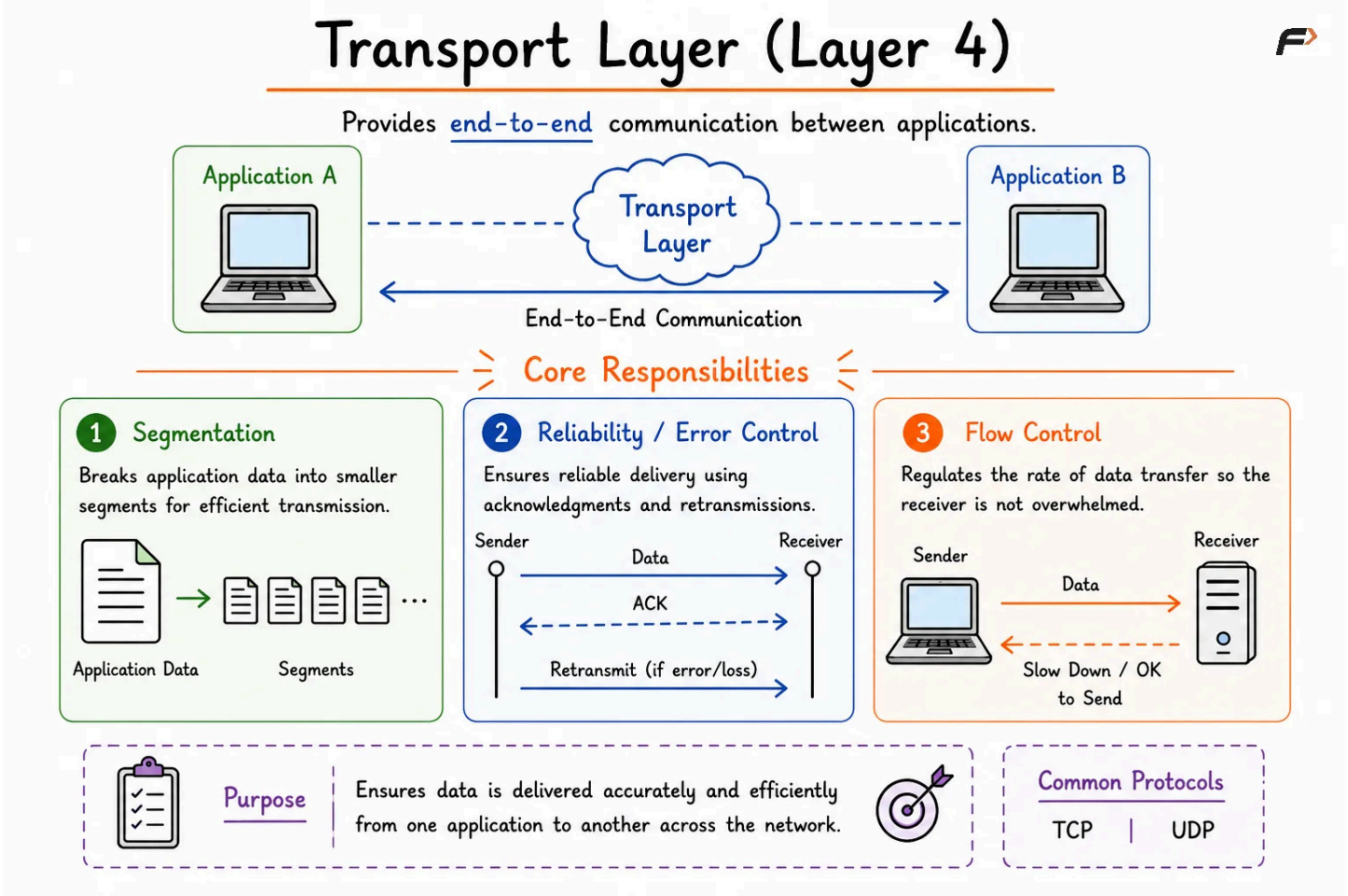
Which data belongs to which file or request

The Session Layer ensures that communication is not just a stream of data, but a **structured and continuous interaction** between systems.

Transport Layer (Layer 4)

The **Transport Layer** is one of the most important layers in real networking. It is responsible for **end-to-end communication between applications**.

While lower layers ensure that data reaches the correct machine, the Transport Layer ensures that the data reaches the **correct application** on that machine and is delivered reliably if needed.



Core Responsibilities

Segmentation: Dividing large data into smaller units called segments.

Reliability / Error Control: Ensuring data is delivered correctly without loss or corruption.

Flow Control: Managing the rate of data transfer between sender and receiver.

Connection-oriented / Connectionless communication: Supporting both reliable and fast transmission modes.

Overall, the Transport Layer **controls the reliability of communication between systems**.

Why Transport Layer is Needed

The network layer (IP) can deliver data to a machine, but it does not know **which application** should receive it. This is what the Transport Layer manages. It uses **port numbers** to identify applications. For example: A browser may use one connection, zoom may use another and whatsApp may use another. All of these run on the same machine, and the Transport Layer ensures that incoming data is delivered to the correct process.

Segmentation

Data received from the Session Layer is divided into smaller units called **segments**, allowing large messages to be broken down into manageable pieces for efficient transmission across the network.

Source and destination port numbers (to identify the communicating applications)

Sequence numbers (to maintain correct ordering of data)

Control information (used for acknowledgments, error handling, and flow control)

This segmentation ensures reliable communication, as data can be transmitted efficiently, retransmitted if lost, and correctly reassembled in the proper order at the destination, even if segments arrive out of sequence.

Flow Control

Flow control ensures that a fast sender does not overwhelm a slow receiver by regulating the rate at which data is transmitted. For example:

Sender (desktop): Capable of transmitting data at a high speed of around 100 Mbps, meaning it can send large amounts of data very quickly.

Receiver (mobile): Limited to a lower processing and receiving speed of around 10 Mbps, meaning it cannot handle data arriving too fast without potential delays or loss.

In such cases, the Transport Layer adjusts the transmission speed and may signal the sender to slow down, ensuring that the receiver can process incoming data efficiently without loss or congestion.

Error Control

During transmission, some data units (segments) may get lost, delayed, or corrupted due to network issues such as congestion, interference, or unreliable links. The Transport Layer ensures reliable delivery using mechanisms like:

Acknowledgements (ACKs): The receiver sends confirmations for successfully received segments, allowing the sender to track which data has arrived.

Retransmissions: If an acknowledgement is not received within a certain time, the sender assumes the segment was lost and sends it again.

Automatic Repeat reQuest (ARQ): A protocol that combines acknowledgements and retransmissions to automatically detect missing or corrupted data and recover it.

Using these mechanisms, the Transport Layer ensures that all data is delivered accurately and in order, even if some segments are lost or arrive incorrectly during transmission.

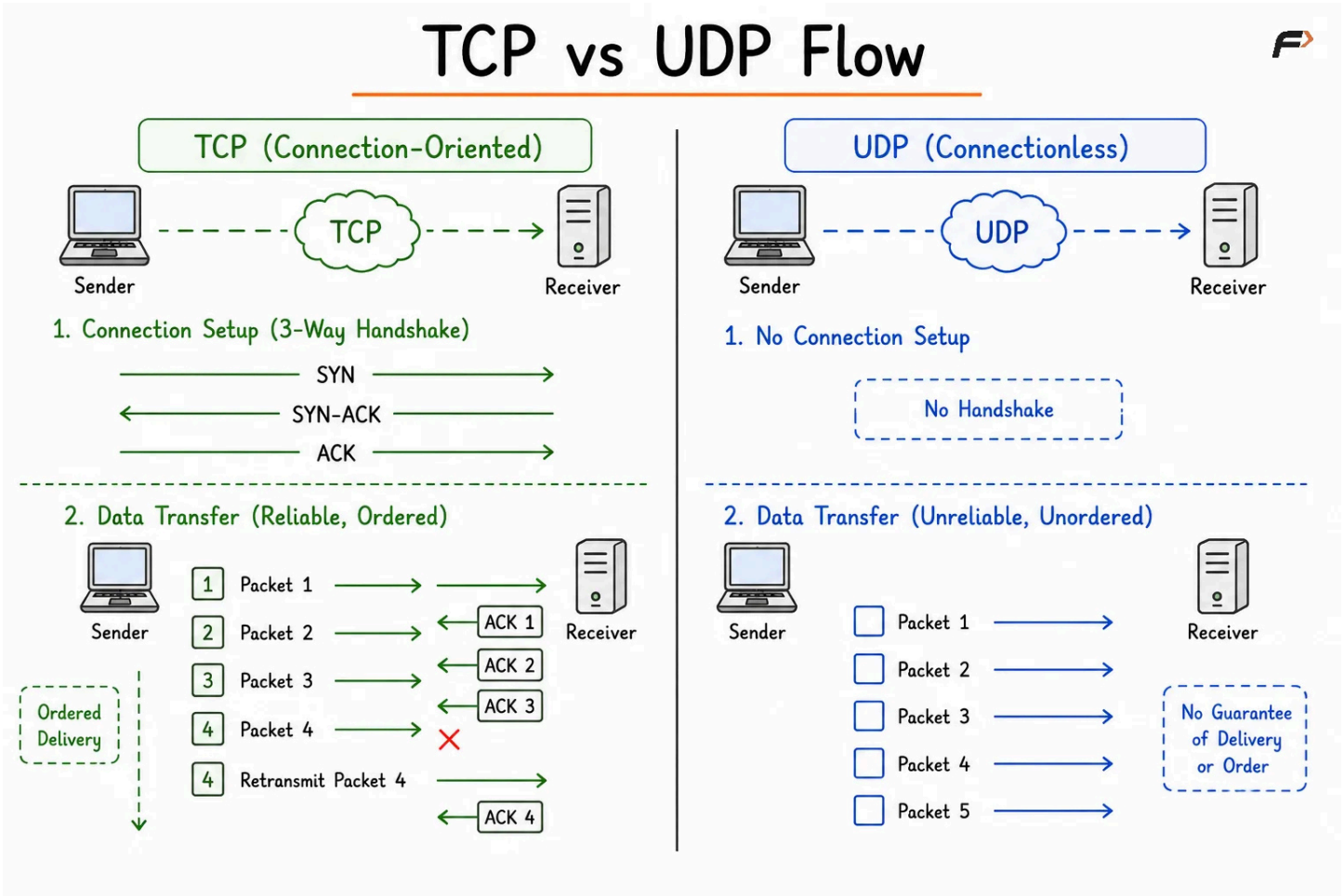
Connection Types

The Transport Layer supports two types of communication:

Connection-Oriented Transmission → TCP Ensures reliable delivery with acknowledgements and retransmissions.

Connectionless Transmission → UDP Sends data without establishing a connection, faster but less reliable.

TCP vs UDP



UDP is generally faster than TCP because it does not establish a connection or wait for acknowledgements, making it suitable for time-sensitive applications.

UDP (User Datagram Protocol):
Connectionless protocol with no guarantee of delivery, ordering, or duplication control.
No feedback or retransmission, resulting in low latency and minimal overhead.
Used in: streaming (audio/video), online gaming, VoIP, real-time communication.

TCP (Transmission Control Protocol):
Connection-oriented protocol that establishes a reliable communication channel before data transfer.
Uses acknowledgements, retransmissions, and sequencing to ensure complete and ordered delivery.
Used in: web browsing (HTTP/HTTPS), email, file transfers.

Reliability: TCP ensures reliable delivery of data, while UDP does not guarantee delivery.

Speed: UDP is faster due to lack of overhead, whereas TCP is slower because of error checking and control mechanisms.

Ordering:
TCP maintains the correct order of data packets, while UDP may deliver packets out of order.

Error Handling: TCP detects errors and retransmits lost data, whereas UDP performs minimal error checking without recovery.

Connection Type: TCP requires a connection setup (3-way handshake), while UDP sends data without establishing a connection.

Use Case Preference: TCP is preferred when accuracy is critical, while UDP is preferred when speed and low latency are more important than reliability.

Main Protocols

The main Transport Layer protocols include: TCP, UDP and QUIC. QUIC (Quick UDP Internet Connections) is a modern transport protocol developed by Google that runs over UDP but provides many of the reliability features of TCP along with built-in encryption similar to TLS. It is designed to reduce latency by combining connection establishment and encryption handshakes into a single step, enabling faster connections compared to traditional TCP + TLS.

QUIC also supports multiplexing multiple streams over a single connection without head-of-line blocking, improving performance for web applications. It forms the foundation of HTTP/3 and is widely used to deliver faster, more secure, and more efficient internet communication.

Process-to-Process Communication

IP can deliver packets from **host to host**, but applications require communication from **process to process**. The Transport Layer bridges this gap by ensuring that the correct process receives the data.

A **process** is a running program on a machine that wants to send or receive network data. Examples include: Chrome, Whatsapp, Zoom and others.

Network Layer (Layer 3)

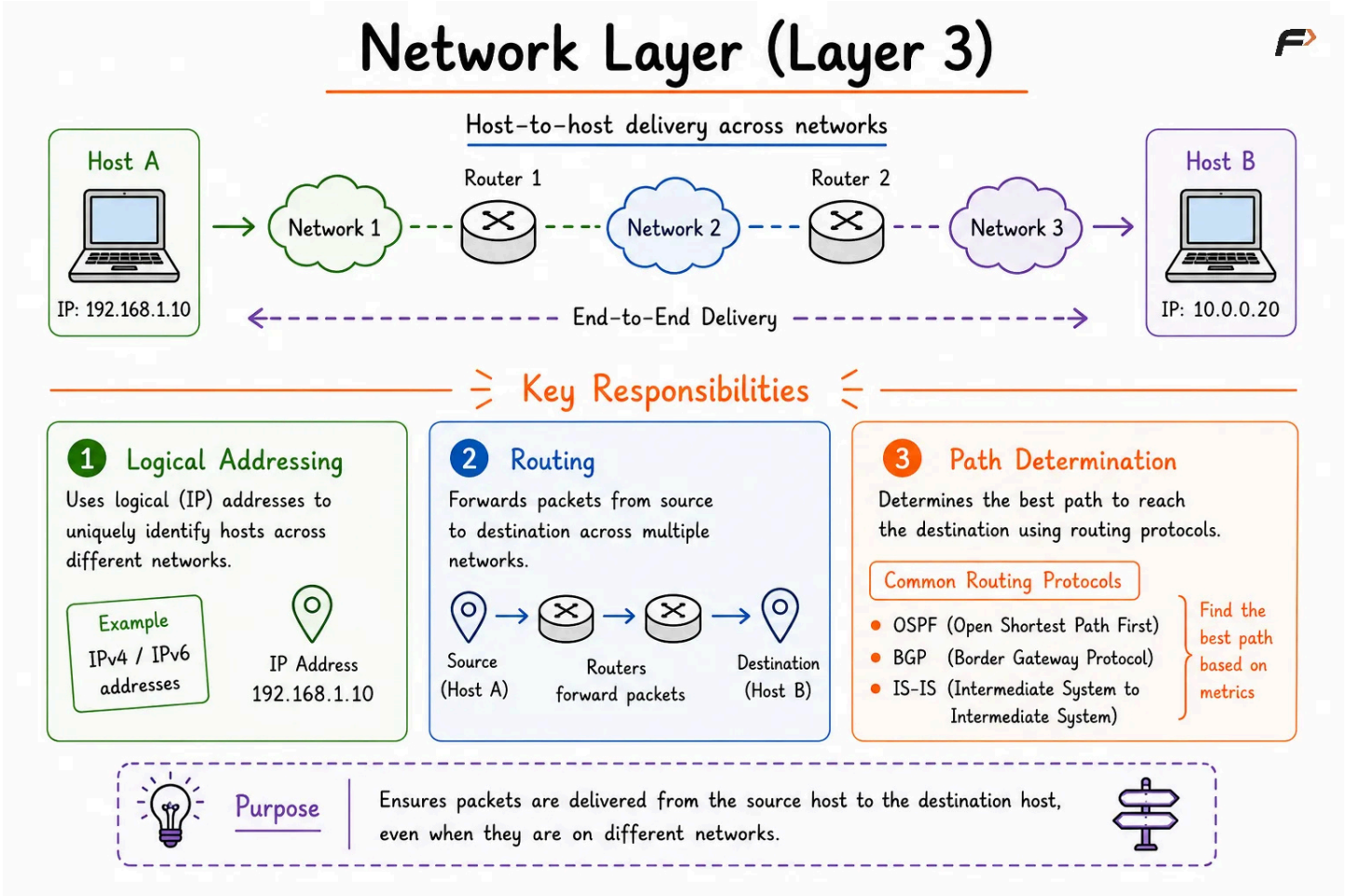
The **Network Layer** is responsible for **host-to-host delivery across networks**. This is the layer where **IP addressing and routing** take place.

It solves a very important problem: **What if the destination is not on the same local network?** In such cases, data cannot be delivered directly. The Network Layer defines **how data travels from one machine to another across different networks**. It works using **IP addresses**. Every packet at this layer contains:

Source IP address

Destination IP address

Routers use this information to forward packets toward their destination.



A common confusion arises between **MAC address** and **IP address**:

MAC Address: Used for **local delivery** within the same network.

IP Address: Used for **wide-area delivery** across multiple networks.

The Network Layer works with IP addresses, not MAC addresses.

Key Responsibilities

Logical Addressing: The Network Layer assigns unique IP addresses (IPv4 or IPv6) to devices, along with subnet masks, enabling identification and communication across different networks. This logical addressing is independent of physical hardware and allows devices on separate networks to locate each other globally.

Routing: The process of forwarding packets from the source to the destination across multiple interconnected networks. Routers examine the destination IP address and use routing tables to decide where to send each packet next, ensuring it moves closer to its final destination.

Path Selection: When multiple routes are available, the Network Layer uses routing algorithms and metrics (such as hop count, latency, bandwidth, and congestion) to determine the most efficient or optimal path for data transmission.

Path Determination (Routing)

When data travels across networks, there are often **multiple possible paths** from source to destination. The Network Layer is responsible for:

Identifying multiple possible paths

Choosing the best path based on routing algorithms

Routers use routing protocols to make these decisions, such as:

OSPF (Open Shortest Path First): A link-state routing protocol used within a single organization (intra-domain), which calculates the shortest path using algorithms like Dijkstra's and quickly adapts to network changes.

BGP (Border Gateway Protocol): The primary routing protocol used between different networks (inter-domain), such as ISPs, which determines the best path based on policies, rules, and network reachability rather than just shortest distance.

IS-IS (Intermediate System to Intermediate System): A link-state routing protocol similar to OSPF, widely used by large service providers due to its scalability and efficiency in handling complex and large-scale networks.

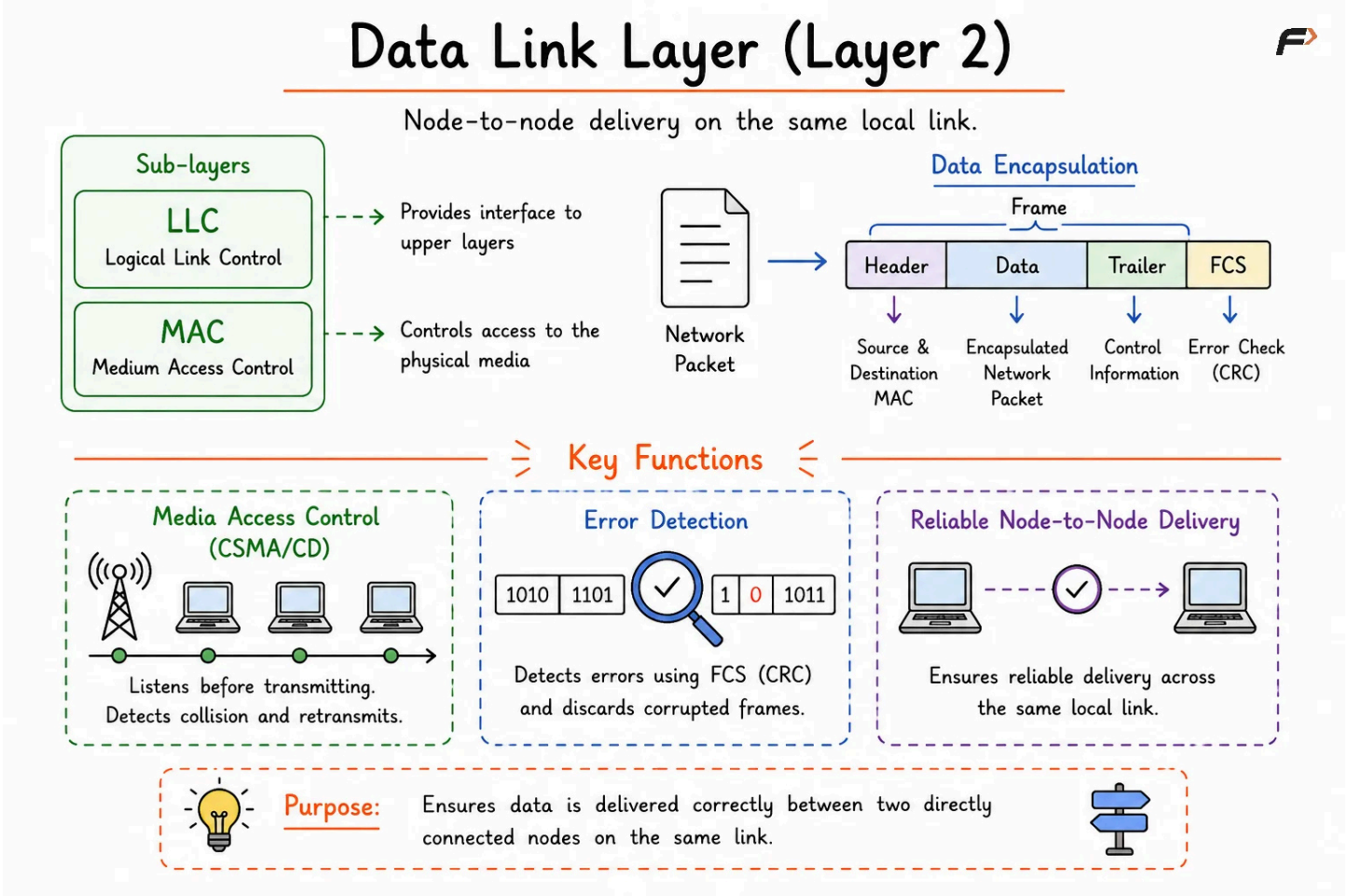
These protocols help determine the most efficient path for packet delivery.

Data Link Layer (Layer 2)

The **Data Link Layer** is responsible for **node-to-node delivery on the same local link**.

It is important to understand that the Data Link Layer is concerned with delivery on the **current link only**, not the entire internet path. It ensures that data is correctly transferred between two directly connected devices. Examples include: Ethernet, Wi-Fi and MAC Layer communication.

Even if the Network Layer (IP) determines that the next hop is some router, the actual delivery on the current network still needs a local mechanism. This is exactly what the Data Link Layer provides.



Data Unit, Sub-layers, and Responsibilities

The data handled at this layer is called an **Ethernet frame**, which is created by encapsulating data with appropriate headers and trailers for transmission over the physical medium. The Data Link Layer is divided into two sub-layers, each responsible for specific functionalities:

MAC (Medium Access Control) Sub-layer: Responsible for accessing the communication medium and deciding how devices share the channel (e.g., Ethernet, Wi-Fi). It also handles **physical addressing (MAC addresses)**, ensures proper framing, and manages collision detection or avoidance mechanisms.

LLC (Logical Link Control) Sub-layer: Responsible for flow control and error control, including mechanisms like **ARQ (Automatic Repeat reQuest)** to ensure reliable transmission

between directly connected devices. It also provides an interface between the Network Layer and the MAC sub-layer.

Data Encapsulation: Wrapping data into frames by adding headers (containing source and destination MAC addresses) and trailers (for error detection, such as CRC).

Media Access Control: Managing how multiple devices access and share the communication medium efficiently, preventing collisions and ensuring fair usage of the network.

Error Detection: Identifying transmission errors using techniques like CRC (Cyclic Redundancy Check) so that corrupted frames can be detected and handled.

Media Access Control (CSMA/CD)

When multiple devices share the same communication medium (like Ethernet), they cannot all send data at the same time.

This is handled using **CSMA/CD (Carrier Sense Multiple Access with Collision Detection)**:

Carrier Sense: Device checks if the medium is free.

Multiple Access: Multiple devices share the same medium.

Collision Detection: If two devices send at the same time, a collision is detected.

When a collision occurs, devices stop, wait for a random time, and retry. The technique used to control access to the medium is called **Media Access Control**.

Types of Addressing

There are two kinds of addressing involved in networking:

Logical Addressing → Network Layer (IP address)

Physical Addressing → Data Link Layer (MAC address)

Given below is a flow of how data evolves across layers:

Transport Layer creates **segments**

Network Layer adds IP addresses and creates **packets**

Data Link Layer adds MAC addresses and creates **frames**

So the final structure becomes: **[MAC1 | MAC2 | IP1 | IP2 | Segment] → Frame**

MAC addresses identify devices on the local network, while IP addresses identify devices across networks.

Collision Scenario

Let us assume multiple devices are connected to a router trying to send data simultaneously.

If two or more devices send data at the same time, a **collision** occurs.

The Data Link Layer monitors this situation.

Using CSMA, devices check if the medium is free before sending.

If busy, they wait and retry.

This ensures fair and organized access to the shared medium.

Key Points

The Data Link Layer works with **frames**

It uses **MAC addresses** for local delivery

Example

Your laptop wants to send traffic to a website, but the destination website is not on your local network (LAN). In this case:

The laptop does **not send the frame directly to the website server**

Instead, it sends the frame to the **default gateway (router)**

The destination MAC address will be the router's MAC address, not the server's MAC

The router then forwards the data further toward the destination.

A common misconception is that an Ethernet frame contains the final destination server's MAC address. In reality:

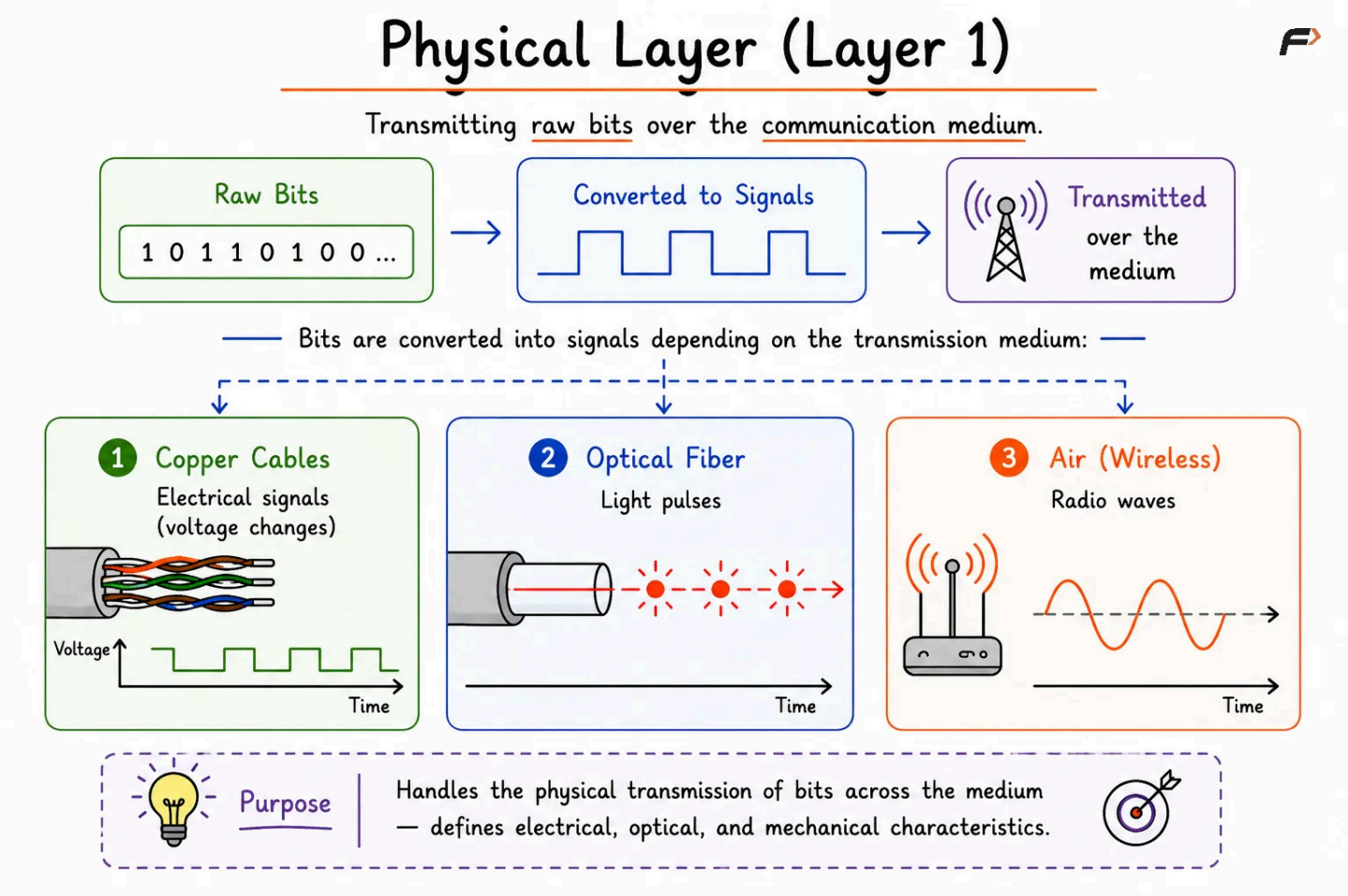
The frame contains the **next-hop MAC address** on the local link

Not the final destination across the internet

This is because MAC addressing is only valid within the local network.

Physical Layer (Layer 1)

The **Physical Layer** is the lowest layer in the OSI model. It is responsible for **transmitting raw bits over the communication medium**.



This layer deals with actual physical aspects such as:

Voltage levels

Electrical signals

Radio waves

Optical pulses

Connectors and cables

Cable standards

Modulation techniques

In simple terms, this layer converts digital data (0s and 1s) into **physical signals** that can travel through a medium.

The physical layer solves a very crucial problem. Ultimately, all higher layer data must be converted into **actual physical signals** for transmission. No matter how structured or meaningful the data is at higher layers, it cannot travel unless it is converted into signals that can move through wires, air, or fiber.

Important Note: Do not mix **frames** and **bits**:

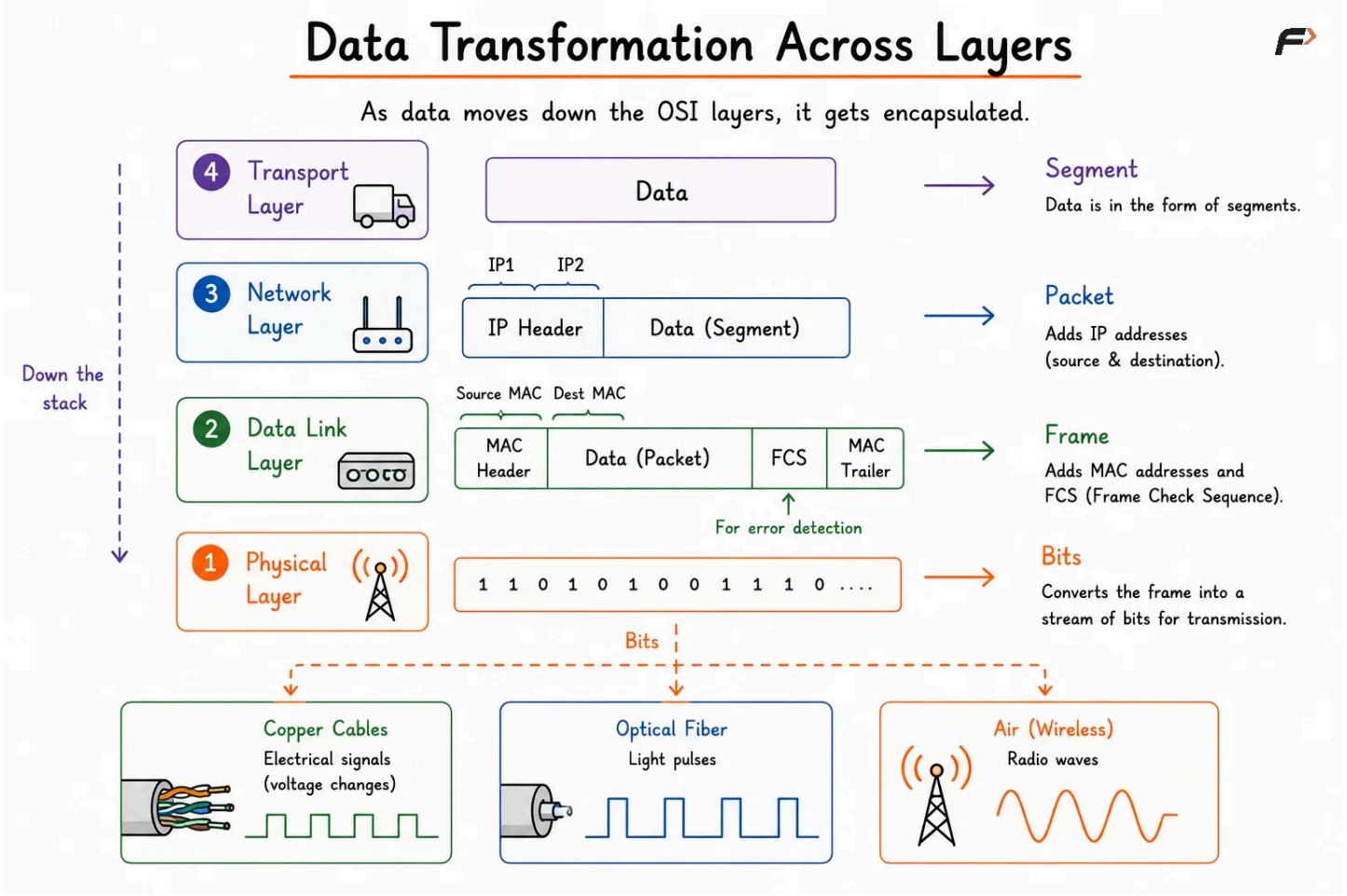
Frames: A concept of the Data Link Layer

Bits: The actual data handled by the Physical Layer

The Physical Layer does not understand frames or packets. It only deals with **bits transmitted as signals**.

Data Transformation Across Layers

The diagram shows how data transforms as it moves down the OSI layers:



Transport Layer: Data is in the form of **segments**

Network Layer: Adds IP addresses and becomes a **packet**

Data Link Layer: Adds MAC addresses and FCS (Frame Check Sequence) and becomes a **frame**

Physical Layer: Converts the frame into a stream of **bits**

The key idea is that each layer adds its own information, and finally, the Physical Layer converts everything into signals for transmission.

The bits are converted into signals depending on the transmission medium:

Copper cables: Electrical signals (voltage changes)

Optical fiber: Light pulses

Air (wireless): Radio waves

The shape and form of the signal depend on the medium being used.

Key Insight

The Physical Layer does not understand meaning, structure, or protocols. It only ensures that bits are **physically transmitted** from one device to another.

How Data Travels Up the Stack on the Receiver Side

When a client sends data, the information moves downward through the OSI layers. Each layer adds its own information before the data is finally transmitted across the network.

However, what happens when the receiver gets this data?

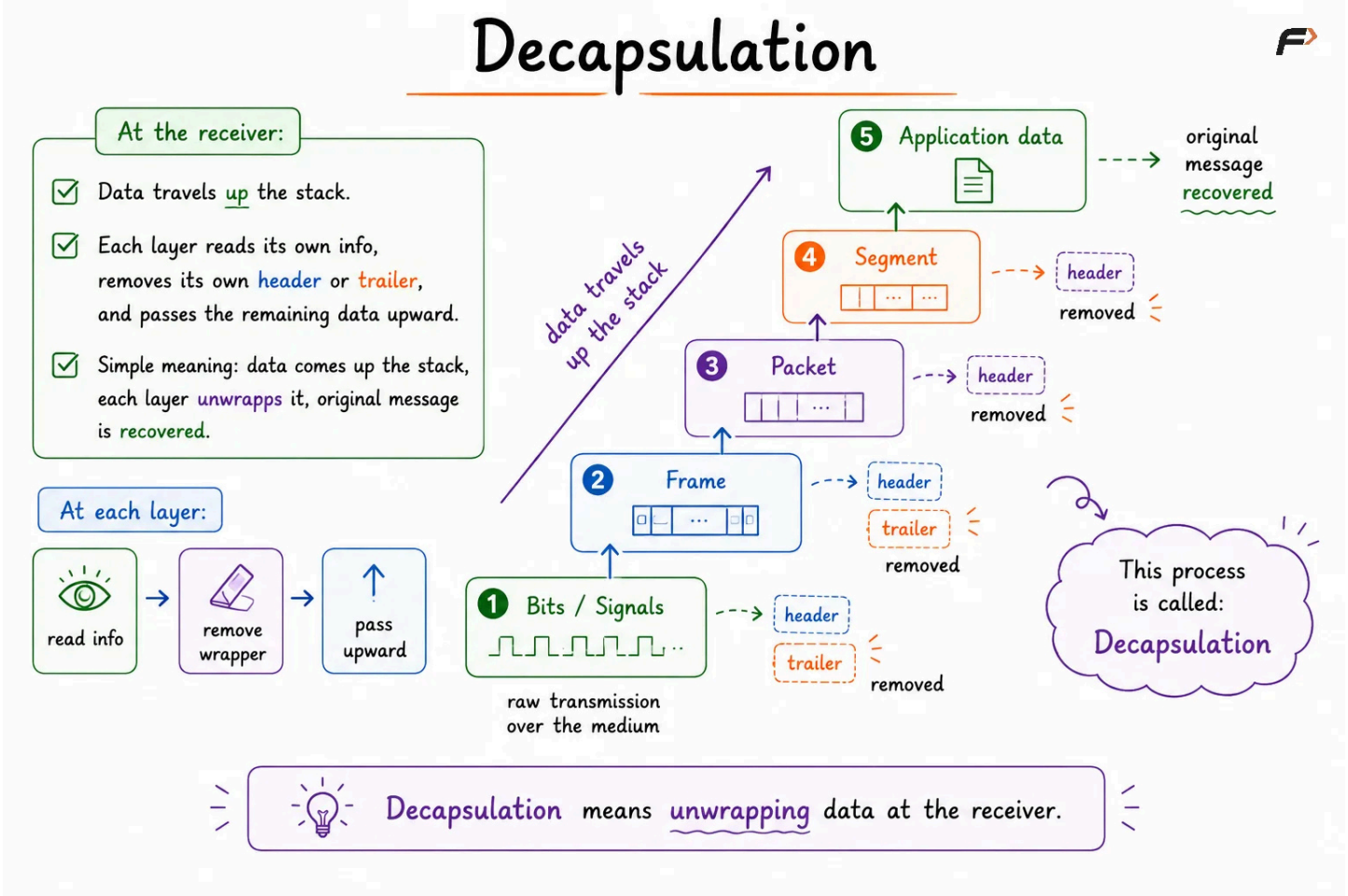
The receiver performs the opposite operation. Instead of adding headers and trailers, it removes them one by one. This process is known as **decapsulation** or **unwrapping the data**.

Thus:

Sender side → Data moves down the stack.

Receiver side → Data moves up the stack.

The responsibilities of the layers remain the same. The only difference is that the receiver removes the information that was previously added by the sender.



Physical Layer

At the receiver side, communication first reaches the Physical Layer in the form of electrical signals, radio waves, or optical signals depending on the transmission medium.

The Physical Layer receives these signals and converts them back into bits.

Thus, the Physical Layer transforms the transmitted signal into digital data that higher layers can process.

Data Link Layer

After the bits are received, they are passed to the Data Link Layer.

The Data Link Layer reconstructs the frame and performs several checks:

Checks the destination MAC address.

Verifies the Frame Check Sequence (FCS) to detect errors.

Removes the frame header and trailer.

After removing the Data Link information, the remaining data is passed upward to the Network Layer.

This process is simply the reverse of what happened at the sender side.

Network Layer

The Network Layer examines the IP packet.

It performs the following tasks:

Checks the destination IP address.

Verifies the protocol field.

Removes the IP header.

Once the IP-related information is removed, the remaining data is forwarded to the Transport Layer.

Transport Layer

The Transport Layer uses the destination port number to determine which application should receive the data.

It also performs several important tasks:

Identifies the correct application.

Checks the order of received data.

Detects missing pieces.

Reassembles data according to sequence numbers.

If some segments are missing, the receiver can request retransmission from the sender.

After proper ordering and reassembly, the data moves to the Session Layer.

Session Layer

A server may communicate with multiple clients simultaneously. Therefore, multiple sessions may exist at the same time.

The Session Layer identifies the correct session and associates the received data with that session.

This ensures that data reaches the proper communication session.

Presentation Layer

The Presentation Layer prepares the data in a form that the application can understand.

It performs operations such as:

Decryption.

Decompression.

Format conversion.

After these operations are completed, the original information becomes available.

Application Layer

Finally, the processed data reaches the Application Layer.

At this stage, the receiver obtains the actual meaningful information.

For example, a web server may receive an HTTP request, or an email application may receive an email message.

Thus, the Application Layer delivers the final message to the application.

Data Flow Direction

The overall direction of data movement can be summarized as:

Sender Side:

Application → Presentation → Session → Transport → Network → Data Link → Physical

Receiver Side:

Physical → Data Link → Network → Transport → Session → Presentation → Application

Therefore, the receiver performs the exact reverse operation of the sender. Instead of encapsulating data, it decapsulates or unwraps the data layer by layer.

Summary

When data reaches the receiver, it travels upward through the OSI model. The Physical Layer converts signals into bits, the Data Link Layer reconstructs frames and checks for errors, the Network Layer processes IP information, the Transport Layer reassembles data and identifies the destination application, the Session Layer matches the correct session, the Presentation Layer performs decryption and decompression, and finally the Application Layer delivers the meaningful message.

Thus, communication on the receiver side is essentially the reverse of the encapsulation process performed by the sender.

TCP/IP Model

The **TCP/IP Model** is the practical model on which the Internet operates. It is named after two important protocols:

Transmission Control Protocol (TCP)

Internet Protocol (IP)

The OSI model is excellent for learning and understanding networking concepts, but the real Internet is practically built around the TCP/IP model.

In simple terms:

OSI Model → Conceptual and educational

TCP/IP Model → Practical and used in real communication

The TCP/IP model focuses more on how networks actually work in practice.

Why Protocols Are Needed

Computers must follow a common set of communication rules in order to communicate successfully.

Without agreed-upon rules, different systems would not understand each other.

Examples of communication rules include:

What is the format of the message?

What is the length of the message?

What is the actual data being sent?

What is the sender's address?

What is the receiver's address?

These communication rules are called **protocols**.

Protocols make communication understandable between different machines and increase interoperability across networks.

Why the Name TCP/IP?

The transport layer can use different protocols such as:

TCP (Transmission Control Protocol)

UDP (User Datagram Protocol)

TCP is commonly used when reliability is important. It helps ensure that lost data can be detected and retransmitted.

UDP is often used when lightweight and fast communication is more important than reliability. For example, in live streaming, losing a few packets is usually acceptable because retransmitting old packets may no longer be useful.

This raises an interesting question:

Why is the model called TCP/IP and not UDP/IP?

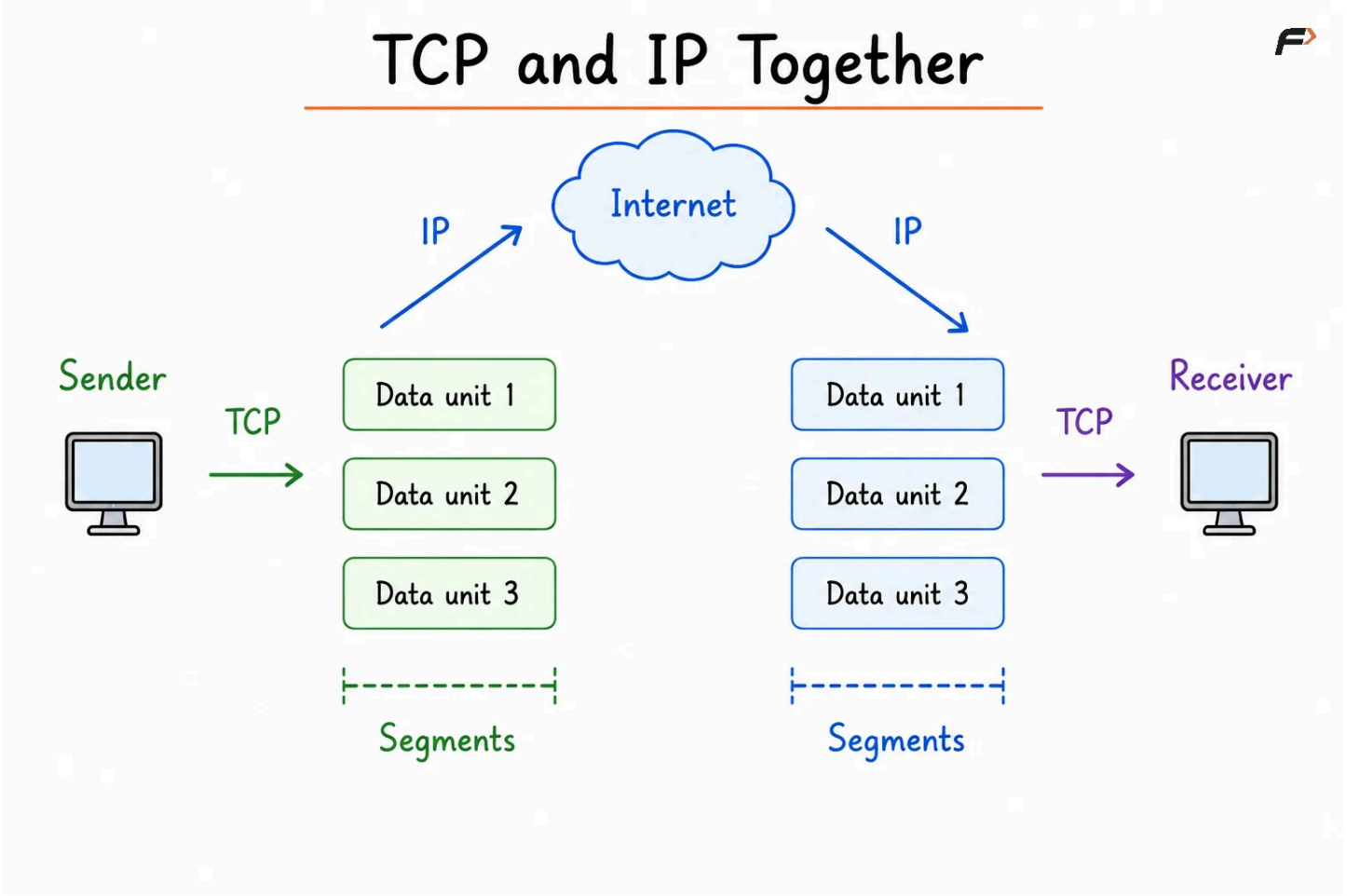
The reason is historical. TCP and IP were the protocols most closely associated with the early Internet. As the Internet evolved, UDP and many other protocols were added, but the name TCP/IP remained.

Therefore, TCP/IP does not mean that only TCP and IP exist.

It actually refers to an entire **protocol suite** containing many protocols that work together.

TCP and IP Working Together

The diagram below shows how TCP and IP cooperate during communication:



The sender uses **TCP** to divide data into smaller units called segments.

These segments are passed to **IP**, which is responsible for delivering them across networks.

The segments travel independently through the Internet.

At the receiver side, **TCP** reassembles the segments in the correct order.

This demonstrates the separation of responsibilities:

TCP: Segmentation, ordering, and reliability.

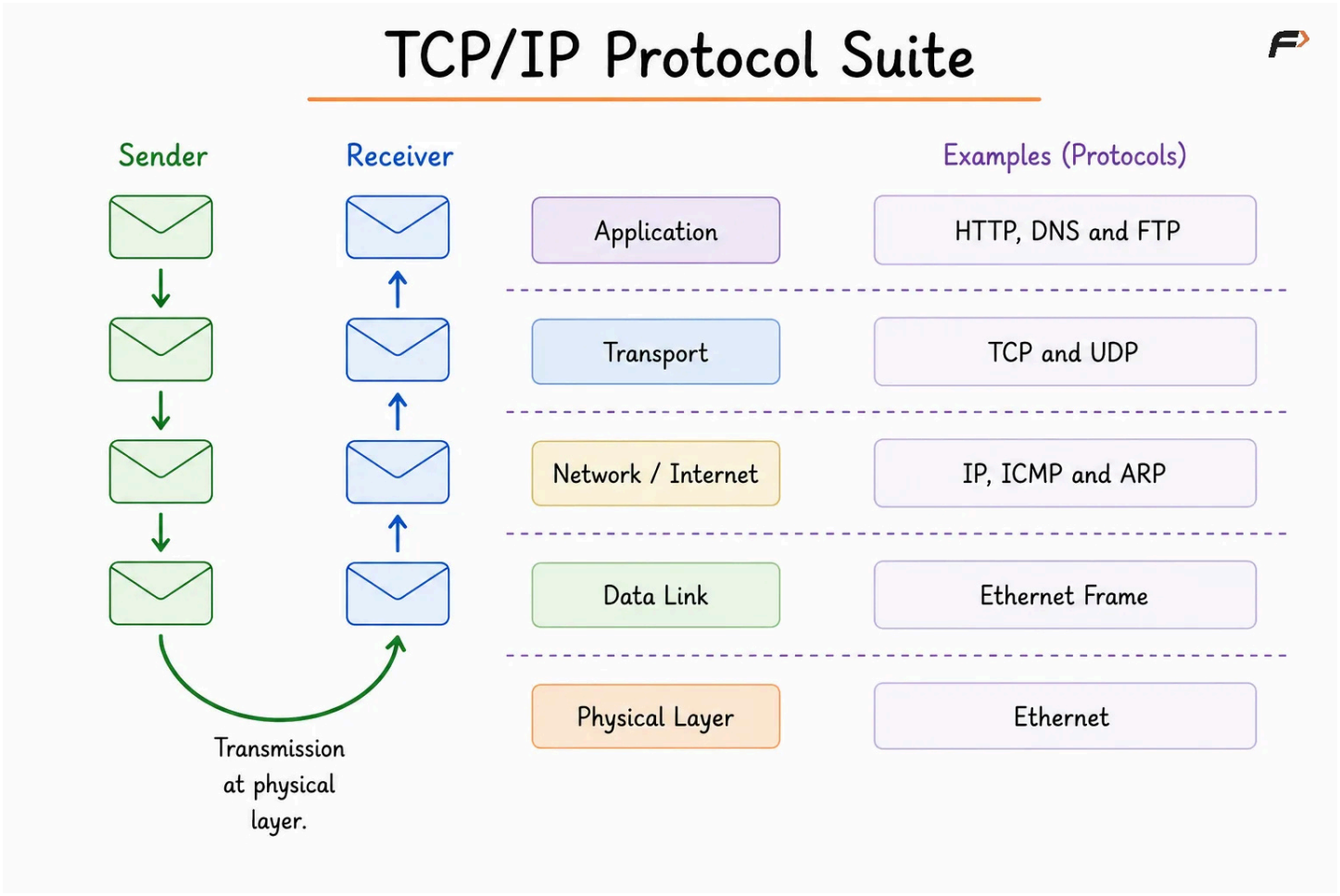
IP: Addressing and delivery.

TCP/IP is a Protocol Suite

TCP/IP is not just two protocols.

It is a collection of multiple protocols organized into layers.

Data moves down these layers at the sender side and moves up the layers at the receiver side.



The TCP/IP model is commonly represented using the following layers:

Application Layer

Transport Layer

Internet Layer

Data Link Layer

Physical Layer

The responsibilities of these layers are very similar to the responsibilities we studied in the OSI model.

OSI to TCP/IP Mapping

The architecture of the TCP/IP model is very similar to the OSI model. The main difference is how responsibilities are grouped.

In the OSI model, we have:

Application Layer

Presentation Layer

Session Layer

Transport Layer

Network Layer

Data Link Layer

Physical Layer

In the TCP/IP model, the Application, Presentation, and Session layers are combined into a single **Application Layer**.

The Transport Layer remains the **Transport Layer**.

The OSI Network Layer becomes the **Internet Layer**.

The Data Link and Physical layers may either remain separate or be combined into a single Link Layer depending on the representation being used.

The functionality remains largely the same. The TCP/IP model simply groups the responsibilities in a more practical manner.

4-Layer and 5-Layer TCP/IP Models

Different books represent the TCP/IP model differently.

Some books use a **5-layer model**:

Application Layer

Transport Layer

Internet Layer

Data Link Layer

Physical Layer

Other books use a **4-layer model**:

Application Layer

Transport Layer

Internet Layer

Link Layer

In the 4-layer model, the Data Link Layer and Physical Layer are combined into a single **Link Layer**.

Both representations are correct because the underlying functionality remains the same.

TCP/IP vs OSI

A useful way to think about the two models is:

OSI Model: Explains networking responsibilities in detail.

TCP/IP Model: Shows how these responsibilities are grouped and implemented in real-world networks.

For example, tasks associated with the Presentation Layer and Session Layer are often handled inside applications themselves. Because of this practical grouping, TCP/IP does not treat them as separate layers.

Summary

The TCP/IP model is the practical networking model used on the Internet today.

It is based on a suite of protocols rather than just TCP and IP alone. TCP provides reliability, while IP provides addressing and delivery across networks.

Compared to the OSI model, TCP/IP groups responsibilities in a more practical way and is typically represented using either four or five layers. Although the layer grouping differs, the core networking responsibilities remain largely the same.

In short, the OSI model helps us understand networking concepts, while the TCP/IP model reflects how real-world Internet communication is actually implemented.

OSI Model vs TCP/IP Model

Below is a side-by-side comparison between the **OSI Model** and the **TCP/IP Model**. It highlights their purpose, structure, and practical relevance in networking.

Aspect	OSI Model	TCP/IP Model
Full Form	Open Systems Interconnection	Transmission Control Protocol / Internet Protocol
Nature	Conceptual Reference Model	Practical Networking Model
Purpose	Designed to explain and standardize networking functions layer by layer.	Designed to enable real internet communication using actual protocols.
Number of Layers	7 layers	4 or 5 layers (depending on representation)
Focus	Best for understanding networking deeply and conceptually.	Best for understanding how the internet actually works in practice.
Implementation	Theoretical model (not directly implemented)	Actually implemented in real-world systems

Summary

The OSI model is a **theoretical framework** designed for understanding networking in layers, while the TCP/IP model is a **practical model** used to implement real-world internet communication.

Data Encapsulation and Decapsulation

Data does not directly travel from an application to the network in its original form. Before it can be transmitted, it must pass through multiple layers of the networking stack. Each layer adds information required for communication, delivery, and processing.

This process of adding information while moving down the network stack is called **encapsulation**. At the receiver side, the reverse process takes place, where each layer removes its information and recovers the original data. This process is called **decapsulation**.

Recalling the OSI Model

Before understanding encapsulation and decapsulation in the TCP/IP model, it is useful to recall the responsibilities of the layers in the OSI model.

Suppose a client wants to send some data. The data moves through the layers in the following order:

Application Layer: Identifies the type of data being sent, such as an HTTP request, email, or other application data.

Presentation Layer: Formats, encrypts, or compresses the data.

Session Layer: Creates and manages the communication session and closes it once communication is complete.

Transport Layer: Handles process-to-process communication and adds port numbers.

Network Layer: Adds IP addressing information.

Data Link Layer: Adds MAC addressing information.

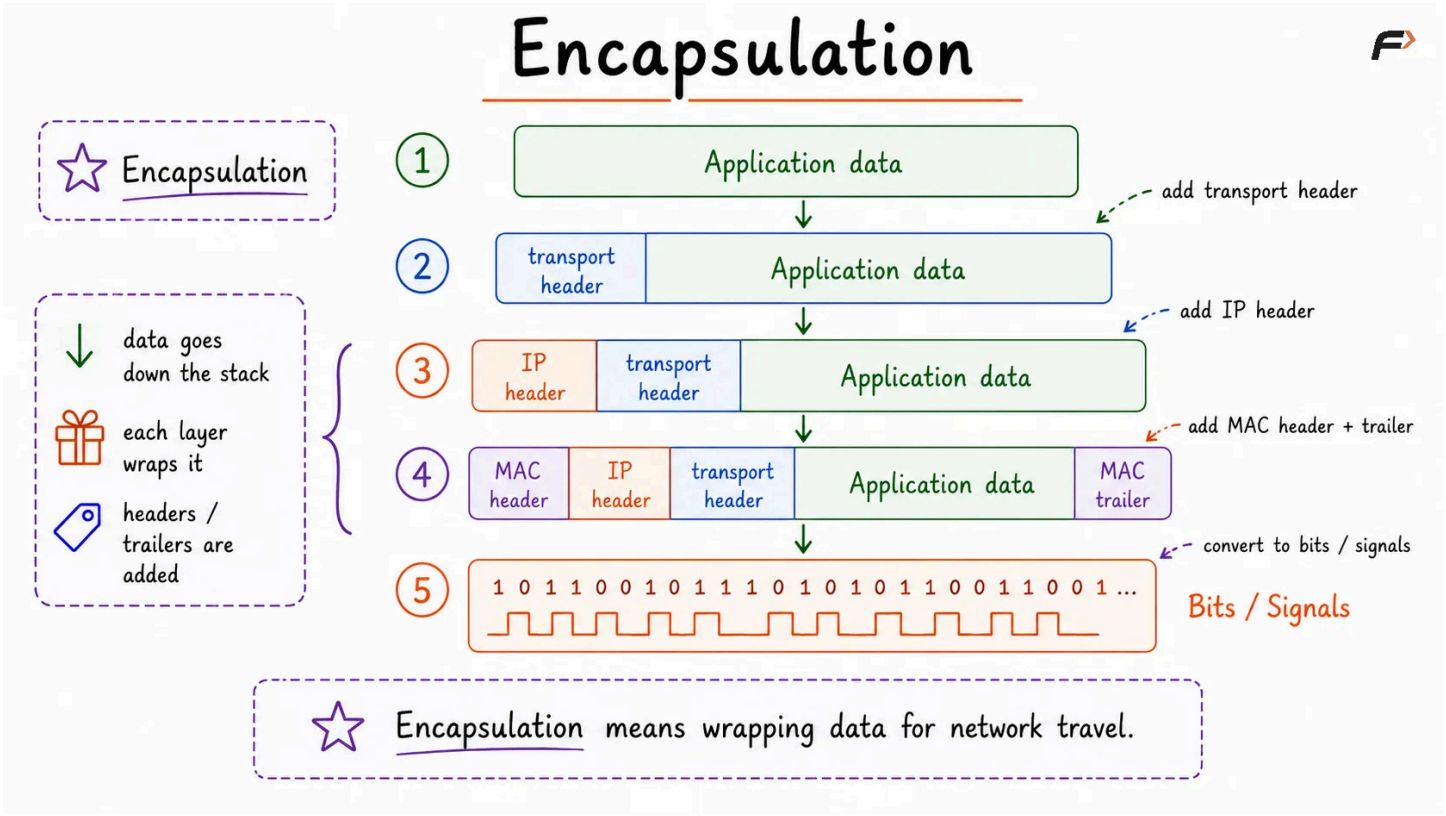
Physical Layer: Converts everything into bits and signals suitable for transmission through the medium.

When data moves down these layers and additional information is added at each step, the process is known as encapsulation.

Encapsulation in the TCP/IP Model

The Internet operates using the TCP/IP model. Encapsulation in the TCP/IP model follows a similar idea: data starts at the top layer and moves downward, with each layer adding information required for communication.

The process begins with the application data. This data may be an HTTP request, an email, a file, or any other application-level information.



Step 1: Application Layer

The application layer generates the actual data that needs to be transmitted.

Examples include:

HTTP request

Email message

Raw file data

Other application data

Step 2: Transport Layer

The transport layer adds transport-related information to the application data.

Examples of information added include:

Source Port Number

Destination Port Number

Sequence Number

After this information is added, the data is ready to move to the next layer.

Step 3: Internet Layer

The Internet layer adds IP addressing information.

Examples include:

Source IP Address

Destination IP Address

These addresses help identify the sender and receiver across networks.

Step 4: Data Link Layer

The Data Link layer adds local network information.

This includes:

MAC Header

Trailer

These additions help communication on the local network link.

Step 5: Physical Layer

Finally, the Physical layer converts everything into bits or signals depending on the transmission medium being used.

At this stage, the original application data has been completely wrapped with information from multiple layers and is ready to travel across the network.

What is Encapsulation?

Encapsulation is the process of wrapping data for network travel.

Data moves down the networking stack, and each layer adds its own information in the form of headers and, in some cases, trailers.

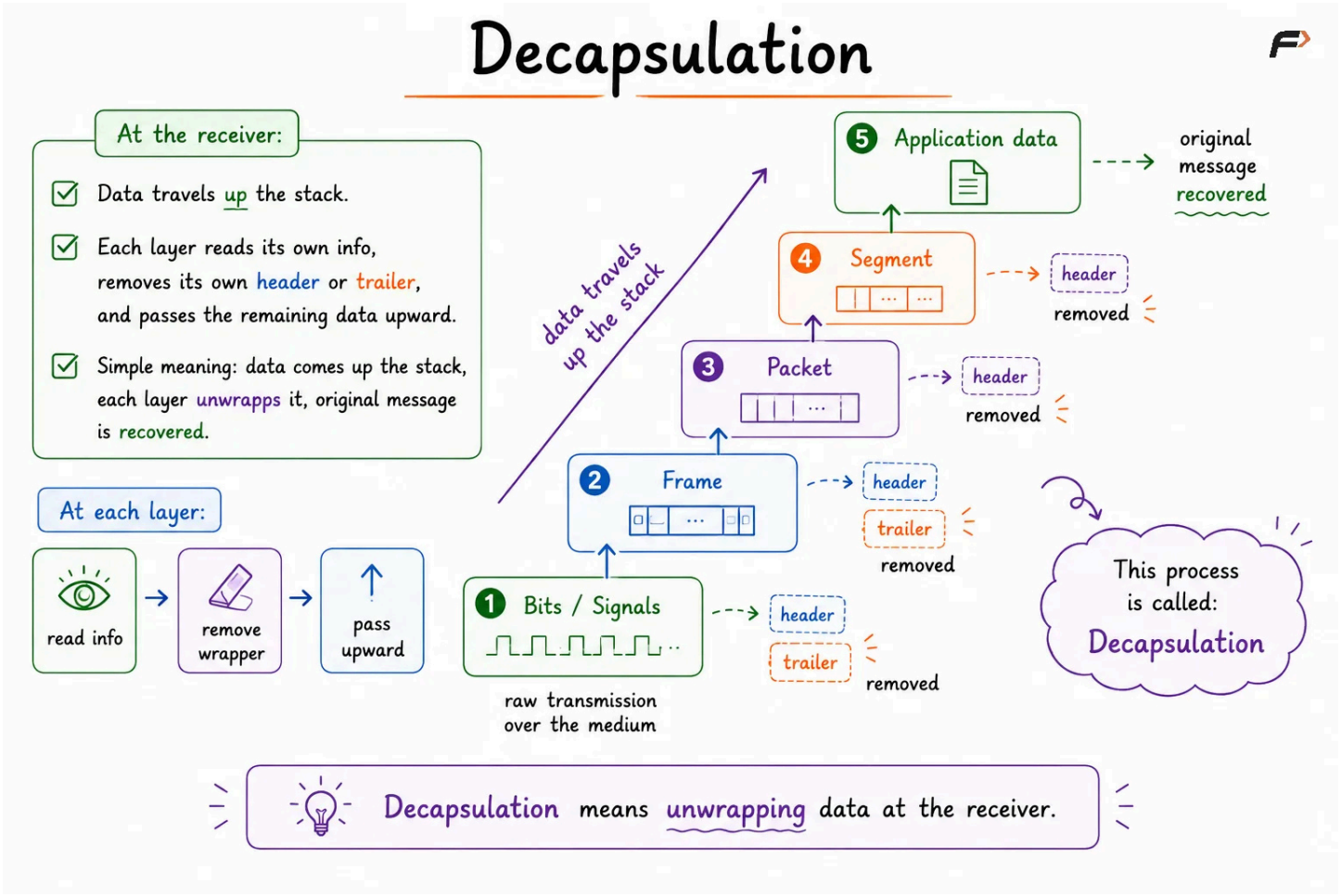
The original application data is gradually transformed until it becomes bits or signals that can be transmitted across the network.

Decapsulation

Once the transmitted data reaches the receiver, the reverse process begins.

The receiver does not immediately see the original application data. Instead, the received bits and signals must move upward through the networking stack.

At each layer, the information added during encapsulation is examined and removed.



Physical Layer

The receiver first obtains the bits or signals from the transmission medium.

Data Link Layer

The Data Link layer processes the received information and removes the MAC-related information and trailers that were added earlier.

Internet Layer

The Internet layer verifies and processes the IP addressing information and removes the IP header.

Transport Layer

The Transport layer examines information such as destination ports and other transport-related details before removing its own header.

Application Layer

Finally, the remaining data is delivered to the appropriate application.

Since the destination port is known, the operating system can determine which application should receive the data.

At this point, the original application data is available exactly as intended.

What is Decapsulation?

Decapsulation is the process of unwrapping data at the receiver.

The received bits or signals move up the networking stack. Each layer reads its own information, removes it, and passes the remaining data to the layer above.

This continues until the original application data is recovered.

During this process, many additional networking operations may also occur, such as validation and reliability-related processing. However, the core idea remains the same: each layer removes its own information and forwards the remaining data upward.

Summary

When data is transmitted across a network, it does not travel in its original application form.

During **encapsulation**, data moves down the networking stack and each layer adds its own headers or trailers, eventually converting the data into bits or signals suitable for transmission.

During **decapsulation**, the receiver performs the reverse process. The data moves up the stack, each layer removes its own information, and the original application message is recovered.

In simple terms:

Encapsulation: Wrapping data for network travel.

Decapsulation: Unwrapping data at the receiver.

Physical Layer

When you hear "networking", you usually think of websites, IP addresses, TCP, or the Internet. These are abstract concepts. But when one laptop communicates with another, the communication first reaches the bottom layer of the network stack, the **Physical Layer**.

If you remember the OSI model or even the TCP/IP model, the Physical Layer is the last layer from the sender's perspective and the first layer from the receiver's perspective.

On the sender side, its responsibility is to convert data into real signals so that it can be transmitted over a medium.

On the receiver side, its responsibility is to convert those signals back into raw data and pass them up the stack.

At the bottom level, the network deals with:

Electrical pulses on copper cables

Light pulses in optical fibers

Radio waves in Wi-Fi

Frames instead of packets

MAC addresses instead of IP routing

Communication at this level is not about the meaning of data. Instead, it is about how bits physically travel from one place to another.

IP Addressing: Tells where the destination machine is located in the larger network.

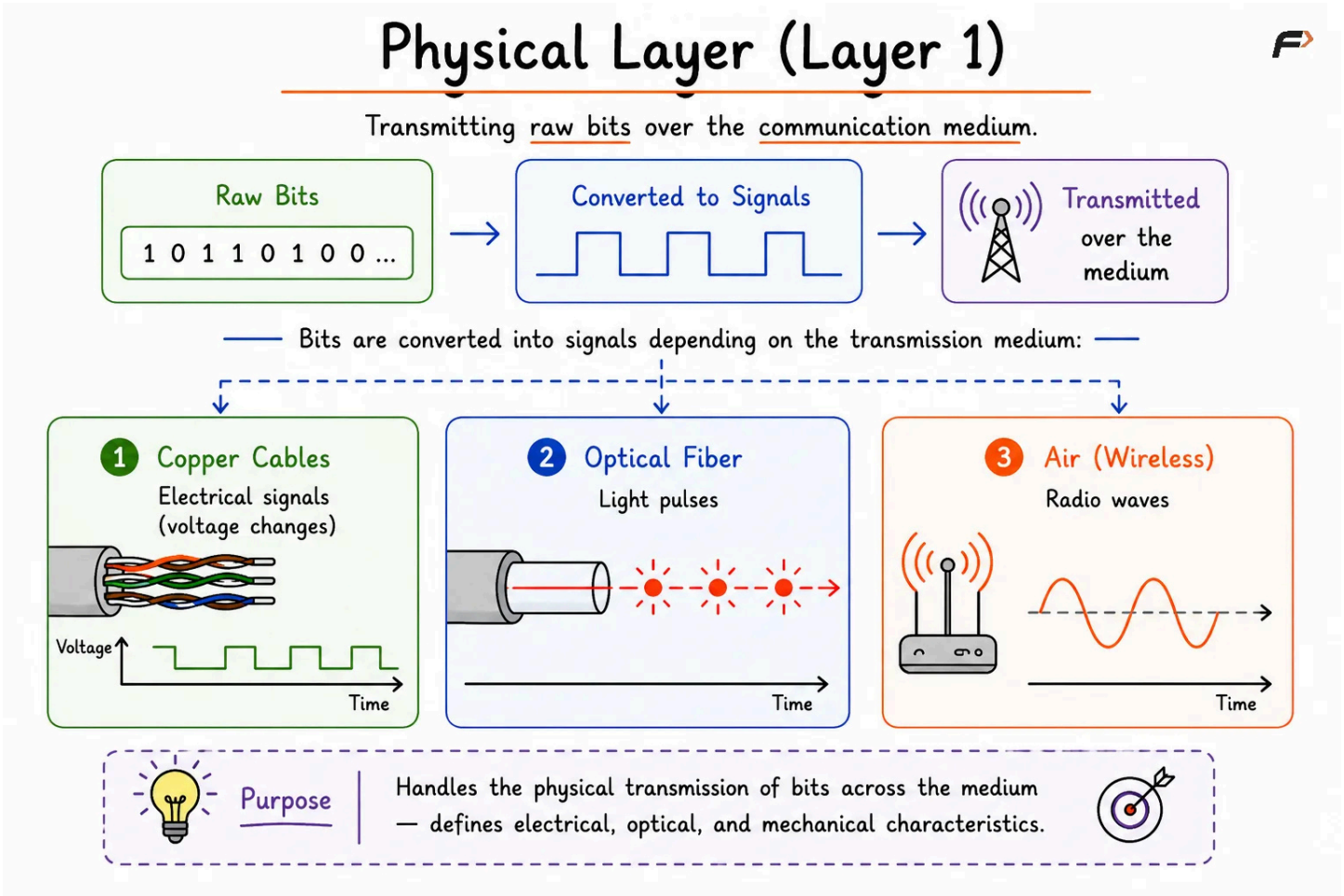
MAC Addressing: Tells where the next hop is on the local link.

Physical Layer: Answers how bits actually travel through the medium.

Depending on the medium being used, bits may travel in the form of electrical pulses, light pulses, or radio waves.

The Physical Layer: What It Actually Does

Machines understand only 0s and 1s. The Physical Layer takes these raw bits and converts them into real signals. These signals are then transmitted through some medium.



The transmission medium could be:

Copper cable

Optical fiber

Wireless medium (Wi-Fi)

The Physical Layer is responsible for answering several important questions:

How is 0 represented?

How is 1 represented?

Which transmission medium should be used?

What signal strength is required?

At what speed should bits be transmitted?

How much can the receiver handle?

Higher layers are responsible for creating and processing data, whereas the Physical Layer is responsible for making that data travel.

In simple terms, the Physical Layer performs the **encoding** and **transmission** of data. It is where abstract information becomes real-world transmission.

Bandwidth vs Throughput

Two important concepts associated with communication are bandwidth and throughput.

Bandwidth

Bandwidth represents the maximum capacity of a communication link to carry data.

It is usually measured in bits per second (bps).

Examples:

100 Mbps

1 Gbps

10 Gbps

When an Internet Service Provider advertises a 100 Mbps or 1 Gbps connection, it is referring to bandwidth.

Throughput

Throughput is the amount of data that you actually get in practice.

For example, even if your Internet plan provides 100 Mbps bandwidth, you may observe actual speeds of:

60 Mbps

70 Mbps

80 Mbps

Throughput can vary because of several factors:

Protocol overhead

Network congestion

Packet behavior

Device limitations

Transmission errors

In simple terms:

Bandwidth = Capacity of the link.

Throughput = What you actually get.

Signal Types

The Physical Layer converts data into real signals. A signal is simply the **physical representation of data**.

Although computers internally work with bits, those bits cannot travel directly. They must first be converted into a physical form.

Depending on the transmission medium, different signal types are used:

Copper cables: Electrical signals

Optical fibers: Light signals

Wi-Fi: Radio signals

Thus, data is converted into a physical form before transmission.

Common Problems During Signal Transmission

While signals travel through the medium, several problems can affect communication.

Cable Cut

If the cable is damaged or cut somewhere in between, communication will fail.

Loose Connection

If the cable is loosely connected to the next device or next hop, signals may not be transmitted properly.

Signal Degradation

Signals become weaker as they travel over long distances. This reduction in signal strength is called signal degradation.

Noise

Signals may experience electromagnetic interference during transmission. This interference is called noise.

Noise can alter the physical representation of data and may lead to communication errors.

Various protocols and techniques are used to overcome these problems, which will be discussed later.

Bad Port

A faulty or damaged port can also cause communication problems. Data may be transmitted, but the receiving device may fail to receive it correctly.

Summary

The Physical Layer is the bottom-most layer of the networking stack and is responsible for converting raw bits into signals and transmitting them through a medium.

It determines how bits are represented, what medium is used, how fast transmission occurs, and how signals are physically carried. Depending on the medium, data may travel as electrical signals, light pulses, or radio waves.

Bandwidth represents the capacity of a communication link, while throughput represents the actual performance observed in practice. During transmission, various issues such as cable cuts, loose connections, signal degradation, noise, and bad ports may affect communication.

In simple terms, higher layers create and process data, whereas the Physical Layer makes that data travel through the real world.

Transmission Medium

A **transmission medium** is the path through which data travels from one device to another. Whenever a sender communicates with a receiver, the data has to travel through some medium to reach its destination.

However, communication from the sender to the receiver does not necessarily happen over a single medium. The complete journey usually consists of multiple hops, and each hop may

use a different medium.

For example, suppose a machine wants to send data to another machine.

The sender may connect to the home router using Wi-Fi or Ethernet cables.

The home router may connect to the Internet Service Provider using copper cables or fiber.

Further communication may pass through several intermediate devices before reaching the receiver.

Thus, the complete path between sender and receiver consists of multiple hops, and every pair of devices may use a different transmission medium.

In simple terms, a transmission medium is the path through which data travels from one device to another, and multiple media may be involved in the complete communication path.

Types of Transmission Media

Transmission media are broadly classified into two categories:

Guided Media

Unguided Media

Guided Media

In guided media, signals travel through a defined physical medium. There is a visible cable path through which the signals propagate.

Examples of guided media include:

Twisted Pair Cable

Fiber Optic Cable

Coaxial Cable

Since the signals follow a physical path, transmission is more controlled.

Unguided Media

In unguided media, signals travel through air without using physical cables. Although the transmission path is not visible, communication still takes place through electromagnetic waves.

Examples include:

Wi-Fi

Cellular networks

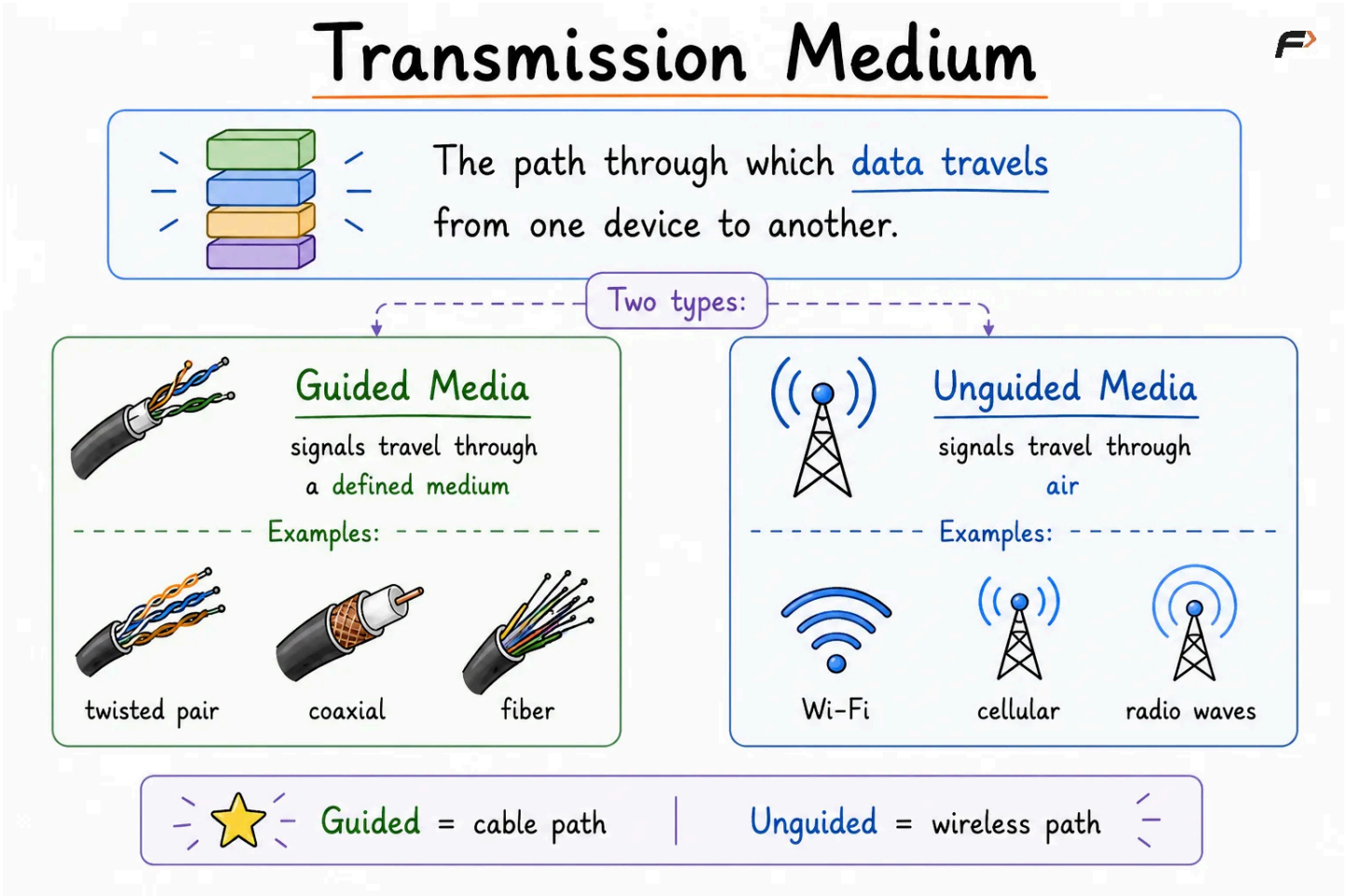
Radio waves

Microwaves

Infrared

Unguided media provides greater mobility and flexibility because there are no cables involved.

For example, if a laptop is connected using a LAN cable, it must remain physically connected to that location. However, a mobile phone connected through Wi-Fi can move anywhere within the Wi-Fi coverage area and still remain connected.



Guided Media vs Unguided Media

Guided Media	Unguided Media
Uses physical cable paths.	No physical cables are involved.
Signals follow a defined medium.	Signals travel through air.
Provides more controlled transmission.	Provides mobility and flexibility.
Examples: Twisted Pair Cable, Fiber Optic Cable, Coaxial Cable.	Examples: Radio waves, Microwaves, Infrared.

In simple terms:

Guided media means a cable path.

Unguided media means a wireless path.

Major Guided Media

The most commonly used guided media are:

Twisted Pair Cable

Fiber Optic Cable

Coaxial Cable

Twisted Pair Cable

Twisted pair cable consists of insulated copper wires twisted together.

It is one of the most commonly used transmission media and is widely used in Ethernet LANs.

The reason behind twisting the wires will be studied separately.

Fiber Optic Cable

Fiber optic cables use light signals for communication.

They are suitable for:

High-speed communication.

Long-distance communication.

However, fiber optic infrastructure is comparatively expensive.

Coaxial Cable

A coaxial cable consists of a central conductor surrounded by shielding.

It is another form of copper cable and is used for carrying high-frequency electrical signals.

Choosing a Transmission Medium

Different types of cables exist because different use cases have different requirements. No single medium is optimal for every situation.

The choice of transmission medium depends on several factors:

Distance requirements.

Speed requirements.

Cost considerations.

Noise and interference.

Available infrastructure.

Depending on these factors, organizations and infrastructure providers decide which transmission medium should be used.

Different cables provide different trade-offs in terms of distance, speed, cost, and resistance to noise, which is why multiple types of guided media exist.

Summary

A transmission medium is the path through which data travels between devices. Communication between a sender and receiver usually consists of multiple hops, and different hops may use different media.

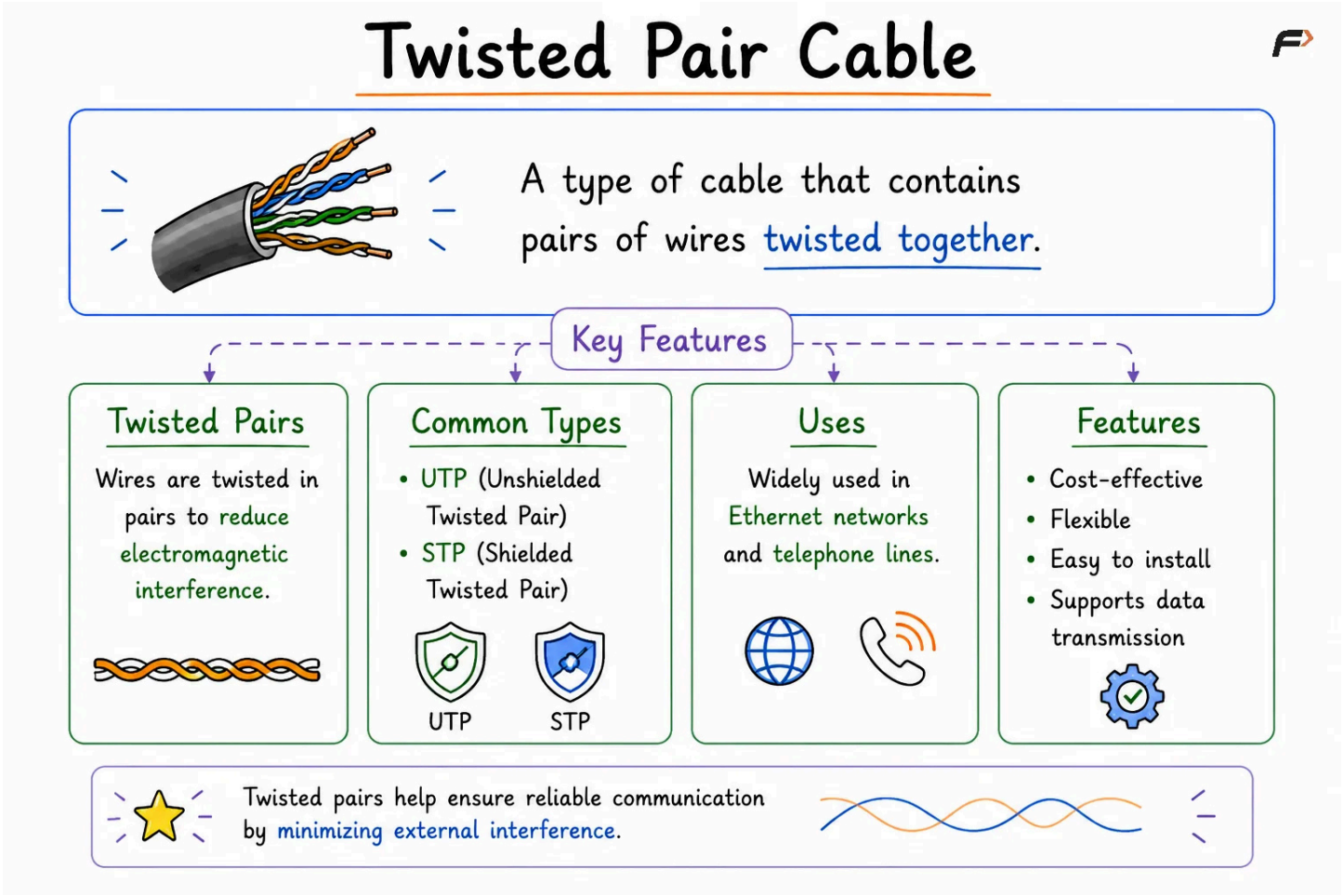
Transmission media are classified into guided media and unguided media. Guided media use physical cable paths, whereas unguided media use wireless transmission through air.

The major guided media are twisted pair cables, fiber optic cables, and coaxial cables. The choice of a particular medium depends on factors such as distance, speed, cost, noise, and infrastructure requirements.

Twisted Pair Cables

Twisted pair cables consist of insulated copper wire pairs twisted together. These cables carry electrical signals and are one of the most commonly used transmission media in Ethernet LANs, homes, and offices.

The twisting helps reduce interference and improves the reliability of communication.



Why are they twisted?

Reduce Electromagnetic Interference (EMI): Twisting helps minimize interference from external electrical signals.

Reduce Crosstalk: Twisting reduces signal leakage between neighboring wire pairs.

Support Noise Cancellation: Both wires experience nearly the same external noise, allowing the receiver to cancel it.

Improve Signal Quality Over Distance: Twisting helps preserve signal quality over longer distances.

Reduce Transmission Errors: Lower interference means fewer corrupted bits.

Increase Reliability: These properties make twisted pair cables practical for networking applications.

Twisting continuously changes the positions of the wires, causing both wires to experience nearly the same unwanted noise. Since both wires see similar interference, the receiver can cancel out most of that common noise.

Noise Cancellation Example

Suppose one wire carries **+1** and the other carries **-1**. The receiver computes the difference:

1 - (-1) = 2

Now assume external noise affects both wires equally.

Wire A = 1.2

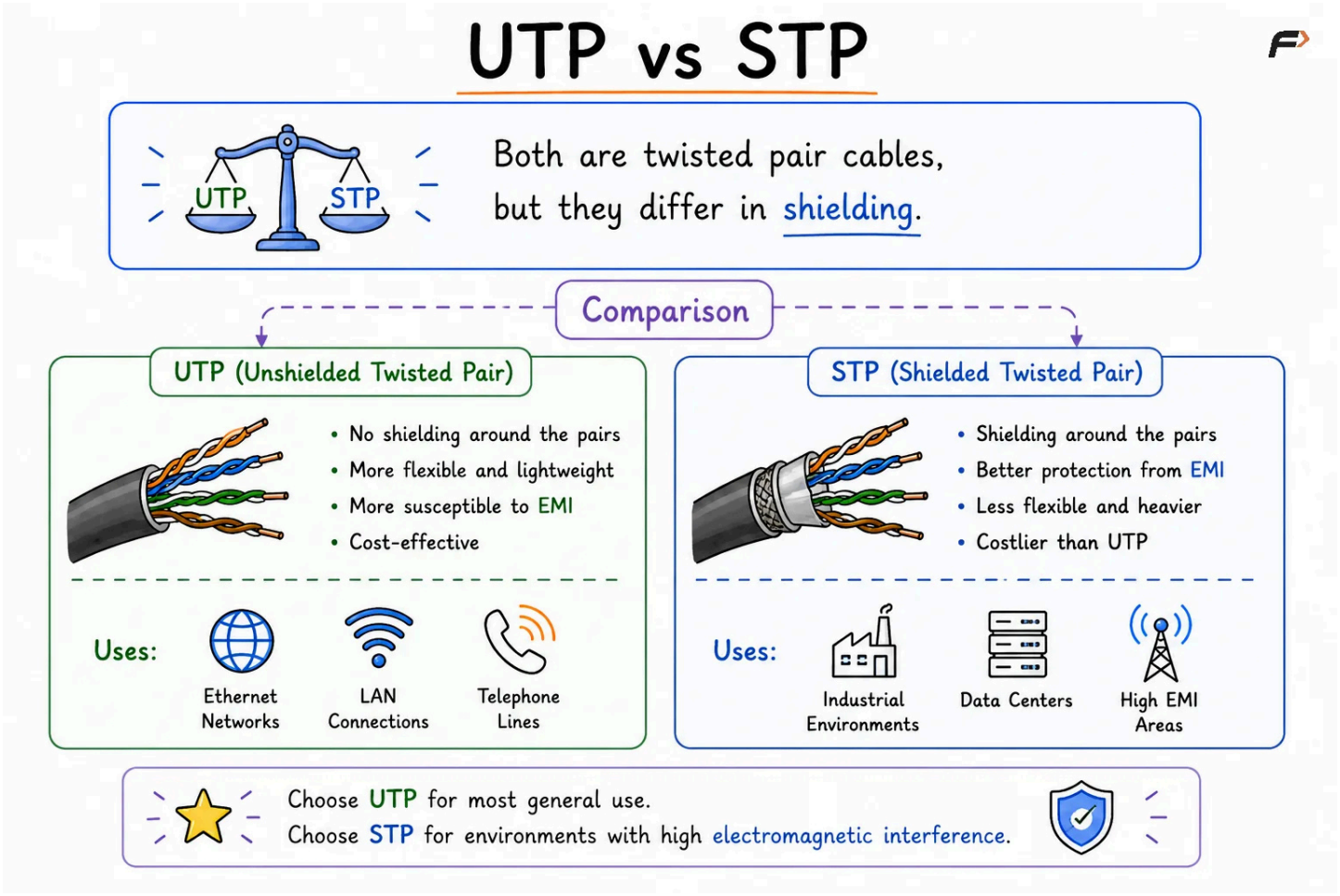
Wire B = -0.8

At the receiver:

1.2 - (-0.8) = 2

Since both wires experience similar noise, the effect of the noise is largely cancelled. This is one of the major reasons why the wires are twisted together.

Types of Twisted Pair



Unshielded Twisted Pair (UTP)	Shielded Twisted Pair (STP)
Does not include extra metallic shielding around the wire pairs and relies mainly on twisting to reduce interference.	Includes metallic shielding for additional protection against electromagnetic interference and crosstalk.
Generally cheaper and commonly used in Ethernet networks.	More expensive because of additional shielding.
Lighter, more flexible, and easier to install.	Heavier and installation is comparatively more difficult.
Grounding is usually not required.	Proper grounding is important for effective shielding.
Suitable for home and office networks.	Often preferred in industrial or high-interference environments.

Common Connectors

RJ11	RJ45
Registered Jack 11, commonly used for telephone connections and voice communication.	**Registered Jack 45**, commonly used for Ethernet and LAN networking.
Typically supports 4 or 6 pins.	Typically uses 8 pins.
Used in telephone systems and DSL connections.	Used with computers, routers, switches, and Ethernet communication.
Smaller connector with fewer conductors.	Larger connector with more conductors, supporting higher-speed communication.

Summary

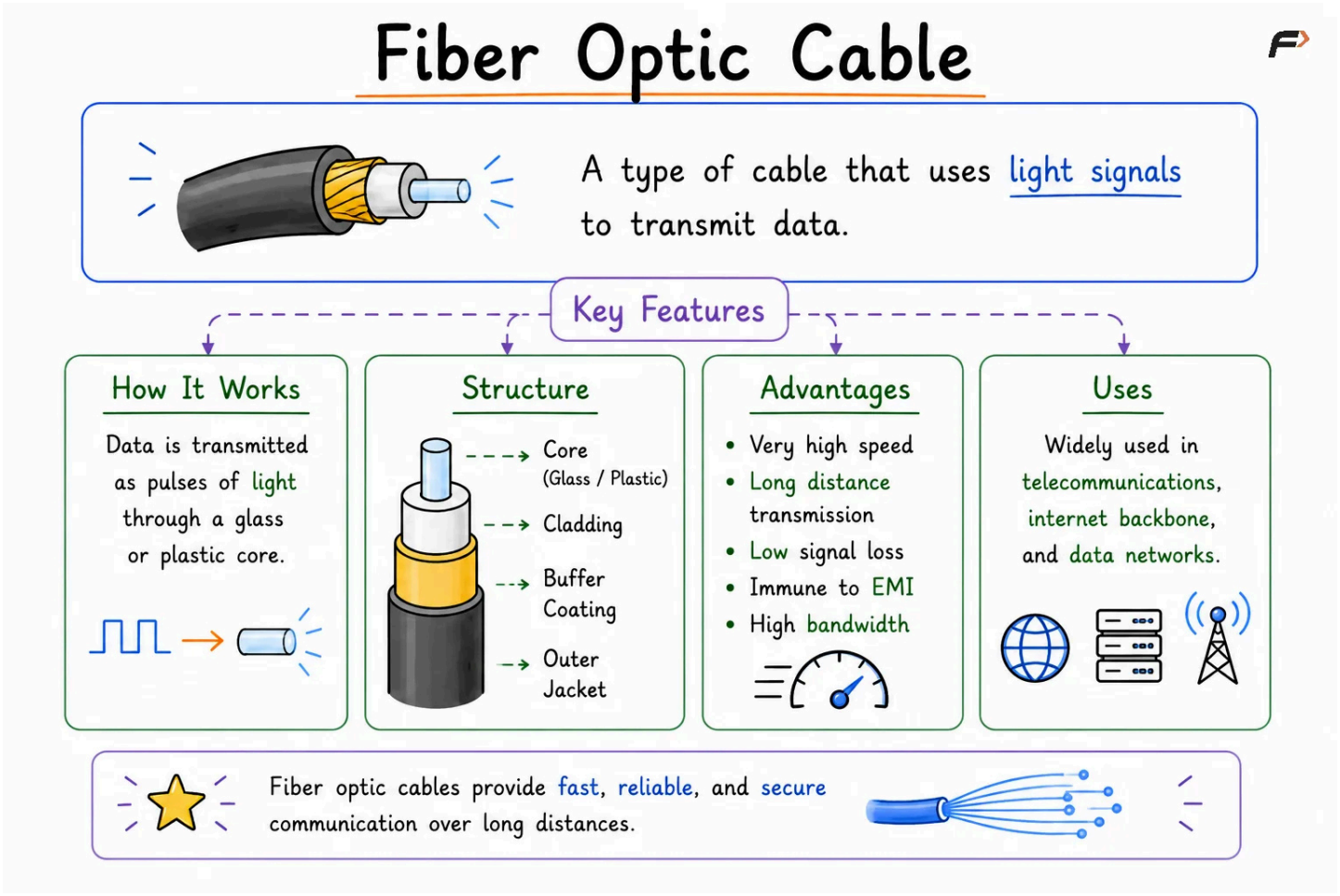
Twisted pair cables consist of insulated copper wires twisted together to reduce interference, crosstalk, and transmission errors. Twisting causes both wires to experience nearly the same noise, allowing the receiver to cancel much of it and improve reliability.

Twisted pair cables are available as UTP and STP. UTP is cheaper and commonly used in homes and offices, while STP provides additional shielding and is preferred in environments with higher interference. Common connectors include RJ11 for telephone communication and RJ45 for Ethernet networking.

Fiber Optic Cable

Fiber optic cables transmit data as light signals through strands of glass or plastic fibers.

The main idea is simple. Data is first converted into light pulses, these light pulses travel through the fiber path, and at the receiver side they are converted back into electrical data that the machine understands.



Why Fiber is Powerful

Fiber is preferred when speed, range, and reliability matter. Compared to copper cables, it offers several advantages:

Very high speed because data is transmitted using light signals.

Low signal loss, allowing communication over very long distances.

High bandwidth, enabling large amounts of data to be carried.

High security since tapping fiber is difficult without disturbing the signal.

Immunity to electromagnetic interference (EMI).

Because of these properties, fiber is commonly used in backbone networks, undersea cables, and long-distance communication links.

Three Main Layers

Layer	What it is	What it does
Core	The innermost glass or plastic part of the fiber.	Carries the light signal used to transmit data.
Cladding	The layer surrounding the core.	Reflects the light back into the core and keeps it traveling forward.
Coating	The outer protective plastic layer.	Protects the fiber from moisture, scratches, and physical damage.

In simple terms:

Core: Carries the light.

Cladding: Keeps the light inside.

Coating: Protects the cable.

Key Features

Very high speed because data is transmitted using light signals.

Long range communication capability.

High bandwidth and ability to carry large amounts of data.

Immune to electromagnetic interference (EMI).

High security because tapping fiber without disturbing the signal is difficult.

Types of Fiber

Single Mode Fiber (SMF)

Single Mode Fiber uses a very small core, allowing light to travel along a single path. It provides high performance and supports long-distance communication. It commonly uses laser light sources and offers very high bandwidth.

Multi-Mode Fiber (MMF)

Multi-Mode Fiber uses a larger core, allowing multiple light paths. It is suitable for shorter distances and provides good performance, although its bandwidth and range are lower than Single Mode Fiber. It commonly uses LED light sources.

Aspect	Single Mode Fiber (SMF)	Multi-Mode Fiber (MMF)
Core Size	Very small core.	Larger core.
Distance	Suitable for long-distance communication.	Suitable for shorter distances.
Bandwidth	Very high bandwidth and performance.	Good but comparatively lower bandwidth.
Light Source	Laser light source.	LED light source.
Cost	More expensive.	Cheaper equipment and lower cost.

Whenever long-distance communication and maximum performance are required, Single Mode Fiber is preferred. For shorter distances and lower cost, Multi-Mode Fiber is commonly used.

Fiber and RF Connectors

Connector	Full Name	Shape / Locking Style	Common Use
SC	Subscriber Connector / Standard Connector	Square push-pull connector	Used in telecom systems and fiber patch panels.
LC	Lucent Connector	Small push-pull connector	Commonly used in switches, routers, and data centers.
ST	Straight Tip	Round connector with twist-lock mechanism	Used in older LANs and campus networks.
FC	Ferrule Connector	Threaded round connector	Used in telecom and long-distance communication environments.
MPO / MTP	Multi-fiber Push-On / Mechanically Transferable Push-On	Rectangular multi-fiber connector	Used in high-speed and high-density data centers.
N-Type	N-Type Connector	Threaded RF connector	Used in antennas, wireless communication systems, and microwave applications.
SMA	SubMiniature Version A	Small threaded RF connector	Used in Wi-Fi equipment, antennas, and RF devices.
TNC	Threaded Neill–Concelman	Threaded RF connector	Used in radio equipment and communication

			systems requiring better vibration resistance

Coaxial Cables

Coaxial cable is a type of guided transmission medium consisting of a central conductor surrounded by insulating and protective layers. It is designed to carry high-frequency electrical signals while reducing interference.

Structure of a Coaxial Cable

A coaxial cable consists of the following components:

Core (Central Conductor): The inner conductor carries the actual data signal and is usually made of copper.

Insulation (Dielectric Layer): This layer surrounds the conductor and provides insulation while maintaining separation between layers.

Metallic Shield: The metallic shield protects the signal from external electromagnetic interference (EMI) and prevents signal leakage outward.

Outer Jacket: The outer layer provides physical protection to the cable.

Electromagnetic Interference (EMI)

Electromagnetic interference refers to unwanted external signals that can distort the original signal. The metallic shielding present in a coaxial cable blocks much of this interference, making coaxial cables more resistant to noise.

In simple terms, the metallic shield protects the signal from outside interference.

Coaxial Cable



A type of cable that uses a central copper conductor surrounded by insulation and shielding.

Key Features

Structure



- Center Conductor (Copper)
- Insulation (Dielectric)
- Shield (Braided / Foil)
- Outer Jacket

How It Works

Carries high-frequency signals with better shielding against electromagnetic interference.



Advantages

- Better shielding than twisted pair
- High signal quality
- Supports higher bandwidth
- More durable



Uses

Used in cable TV, broadband internet, and CCTV systems.





Coaxial cables are reliable and ideal for transmitting high-frequency signals over moderate to long distances.

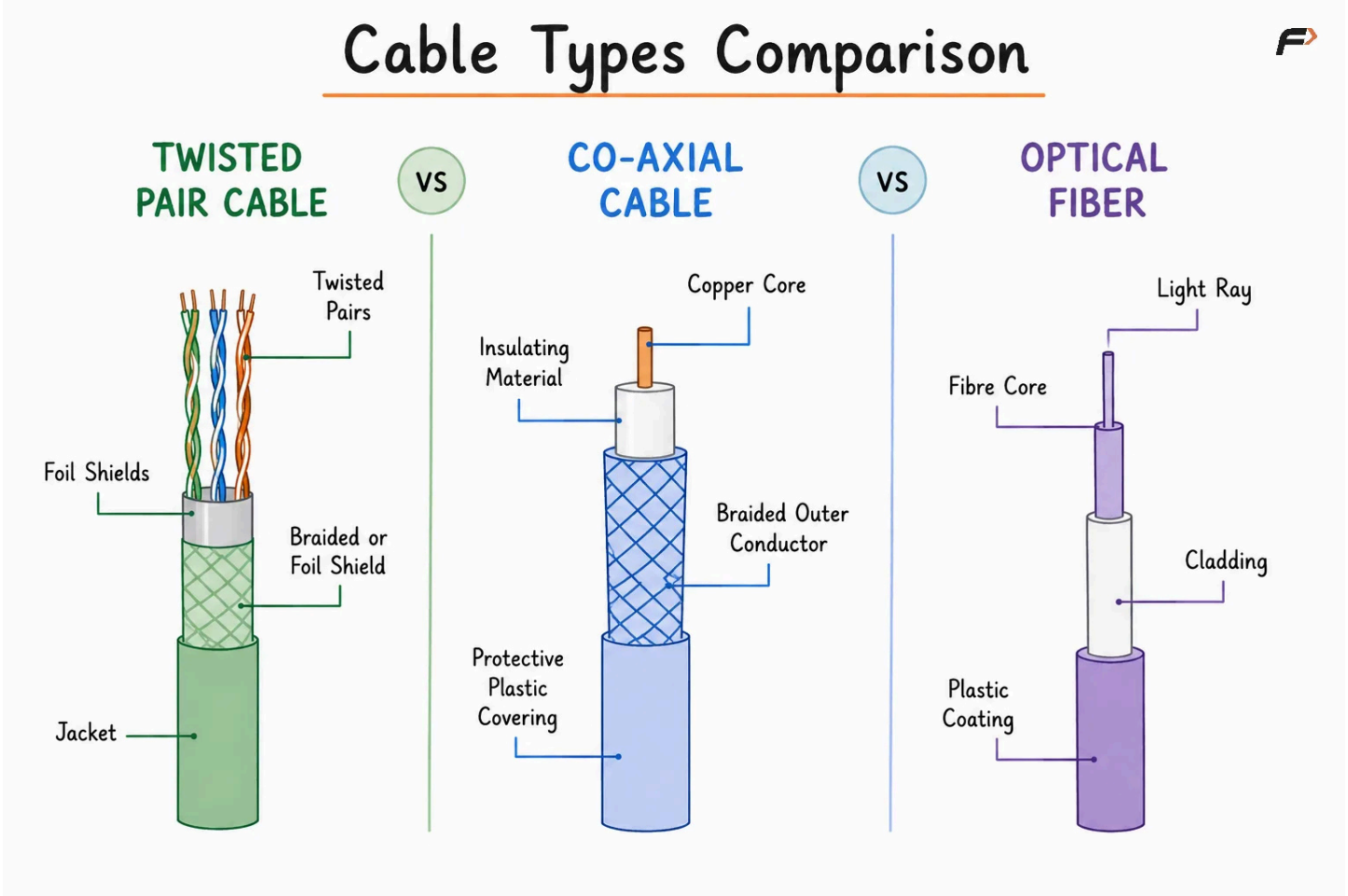


Common Coaxial Connectors

Connector	Full Name	Locking Style	Common Use
BNC	Bayonet Neill-Concelman	Twist-lock	Older networking, CCTV, and radio systems.
F-Type	F-Connector	Threaded / Screw-on	Cable TV, broadband internet, and set-top boxes.
N-Type	N-Connector	Threaded	RF systems, outdoor systems, and wireless equipment.
SMA	SubMiniature Version-A	Threaded	RF devices, antennas, and Wi-Fi equipment.
TNC	Threaded Neill-Concelman	Threaded	RF and communication systems.

These connectors are commonly used in RF systems, cable TV networks, broadband internet installations, and older networking environments.

Comparison of Network Cables



Feature	Twisted Pair (Ethernet)	Fiber Optics	Coaxial Cable
Transmission Mode	Electricity (Electrons)	Light (Photons)	Electricity (Electrons)
Bandwidth (Speed)	Medium (1 Gbps - 10 Gbps)	Very High (100 Gbps+)	Low to Medium
Maximum Distance	Short (~100 m)	Long (10–80+ km)	Medium (~500 m)
Cost	Low	High	Moderate
Interference (EMI)	Low to Moderate	Very High Immunity	High Resistance (Shielded)
Primary Use	Office and Home LANs	Backbone and Long Distance Communication	Cable TV and Broadband Internet

Choosing the Right Cable

The choice of cable depends on:

Required speed and bandwidth.

Distance over which communication is needed.

Budget constraints.

Amount of interference present in the environment.

Although fiber offers the highest speed and best performance, it is also the most expensive. Therefore, many networks continue to use twisted pair or coaxial cables depending on requirements and cost.

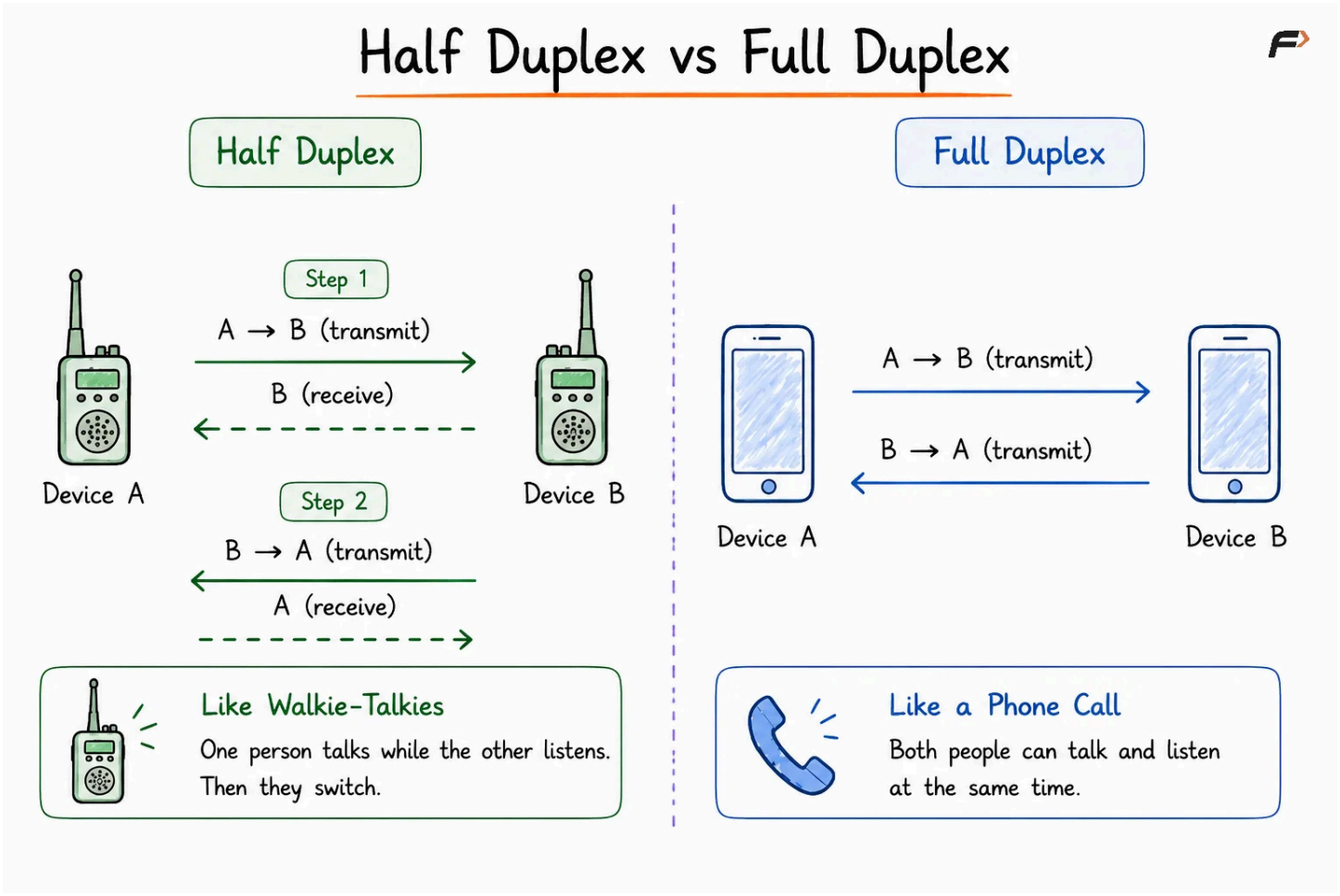
Full Duplex vs Half Duplex

Duplex communication describes how data is transmitted between two devices. It defines whether communication can happen in one direction at a time or in both directions simultaneously.

Duplex communication is primarily divided into two types:

Half Duplex

Full Duplex



Half Duplex Communication

In half duplex communication, data can travel in both directions, but only one side is allowed to transmit at a time. Communication happens in turns, meaning a device can either send or receive data, but not both simultaneously.

This is similar to a walkie-talkie where one person speaks while the other listens.

Collision Possibility

In a shared communication medium, multiple devices may check whether the channel is free. If two devices find the line free at the same instant and both start transmitting, their signals can collide. Hence, collisions are possible in half duplex communication.

If the channel is busy, devices must wait until it becomes free. This waiting leads to lower performance.

Characteristics

Supports communication in both directions.

Only one direction is active at a time.

Devices take turns transmitting.

Collisions are possible.

Performance is comparatively lower due to waiting.

Examples

Walkie-talkies.

Older Ethernet networks using hubs.

Shared communication media.

Full Duplex Communication

In full duplex communication, data can travel in both directions simultaneously. Both devices are allowed to send and receive data at the same time.

Since both directions are active simultaneously, there is no need to wait and collisions do not occur.

Characteristics

Simultaneous two-way communication.

Both directions can be active at the same time.

No collisions occur.

Provides higher performance.

Examples

Modern switched Ethernet networks.

Comparison Table

Aspect	Half Duplex	Full Duplex
Data Flow	Both directions, but only one at a time	Both directions simultaneously
Communication Style	Devices take turns	Both devices communicate together
Collision	Possible	Not possible
Performance	Lower due to waiting	Higher as no waiting is required
Examples	Walkie-talkies, hub-based Ethernet	Modern switched Ethernet

Summary

Half duplex communication allows data to travel in both directions, but only one direction can be active at a time. Devices take turns communicating, which can lead to collisions and lower performance.

Full duplex communication allows both devices to transmit and receive simultaneously. Since there is no waiting, it provides higher performance and avoids collisions.

MAC Addresses: What they are and how they differ from IP

A MAC Address (Media Access Control Address) is the **local hardware address** used at the Data Link Layer. Every network device has a MAC address and it is responsible for delivering frames on the current local network.

A MAC address is used for **hop-to-hop communication**, whereas an IP address is used for **end-to-end communication**.

IP address tells the packet where the final destination machine is located.

MAC address tells the frame which device on the current link should receive it.

A MAC address is typically **48 bits (6 bytes)** long and is usually written in hexadecimal form, for example:

00:1A:2B:3C:4D:5E

Important: IP is end-to-end, whereas MAC is hop-to-hop. The destination IP generally remains unchanged throughout the journey, but the destination MAC address keeps changing at every hop.

Why Does MAC Address Change at Every Hop?

Suppose device A wants to communicate with device B, and there are two routers in between.

A → Router 1 → Router 2 → Device B

Initially, device A already knows the destination IP address of device B. However, the frame cannot directly reach device B. It must first reach Router 1.

Initially, the destination MAC address is the MAC address of **Router 1**.

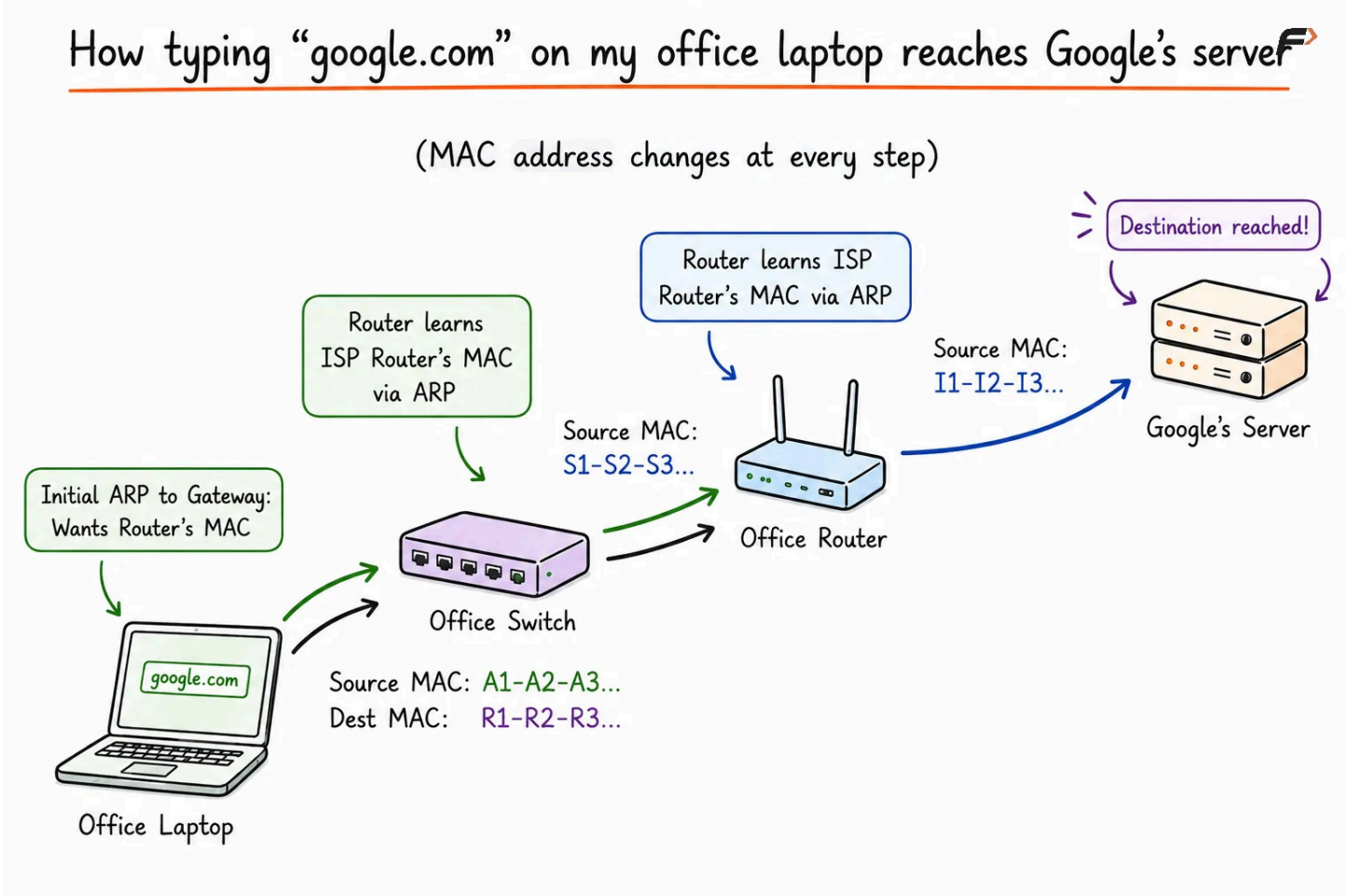
Once the frame reaches Router 1, a new frame is created and the destination MAC address becomes the MAC address of **Router 2**.

After reaching Router 2, the destination MAC address is changed again to the MAC address of **Device B**.

This is why we say that **MAC addresses change at every hop**, whereas the destination IP address remains the same.

How typing "google.com" reaches Google's Server

The diagram below illustrates how MAC addresses change at every hop while the destination IP address remains unchanged.



The laptop wants to communicate with a remote server and knows its destination IP address.

It first performs an **ARP request** to obtain the MAC address of its default gateway.

The first frame is created with:

Source MAC: Laptop's MAC address

Destination MAC: Router's MAC address

The switch forwards the frame based on MAC addresses.

At the router, the old frame is removed and the IP packet is examined.

A new frame is created for the next hop with new source and destination MAC addresses.

This process repeats across multiple routers until the packet reaches the destination network.

Finally, the destination MAC address becomes the MAC address of the server itself.

ARP (Address Resolution Protocol)

ARP is used in IPv4 networks to map an IP address to its corresponding MAC address. Before sending a frame, a device uses ARP to discover the MAC address of the destination device or the default gateway.

NDP (Neighbor Discovery Protocol)

NDP (Neighbor Discovery Protocol) is the IPv6 equivalent of ARP and performs address resolution in IPv6 networks.

Ethernet Frames

An Ethernet frame is the **basic unit of data transmission in Ethernet networks**. Devices inside a LAN exchange Ethernet frames.

At the Network Layer, we typically have an IP packet containing:

Source IP Address

Destination IP Address

Data / Payload

When the packet reaches the Data Link Layer, additional information is added to it.

The Data Link Layer adds:

Source MAC Address

Destination MAC Address

Frame Check Sequence (FCS)

After adding this information, the IP packet is converted into an **Ethernet Frame**, which travels across the network link.

Purpose of Ethernet Frames

Encapsulate upper-layer data.

Identify source and destination devices on the current local network.

Organize LAN communication.

Provide error detection using FCS and CRC.

At a high level, an Ethernet frame contains:

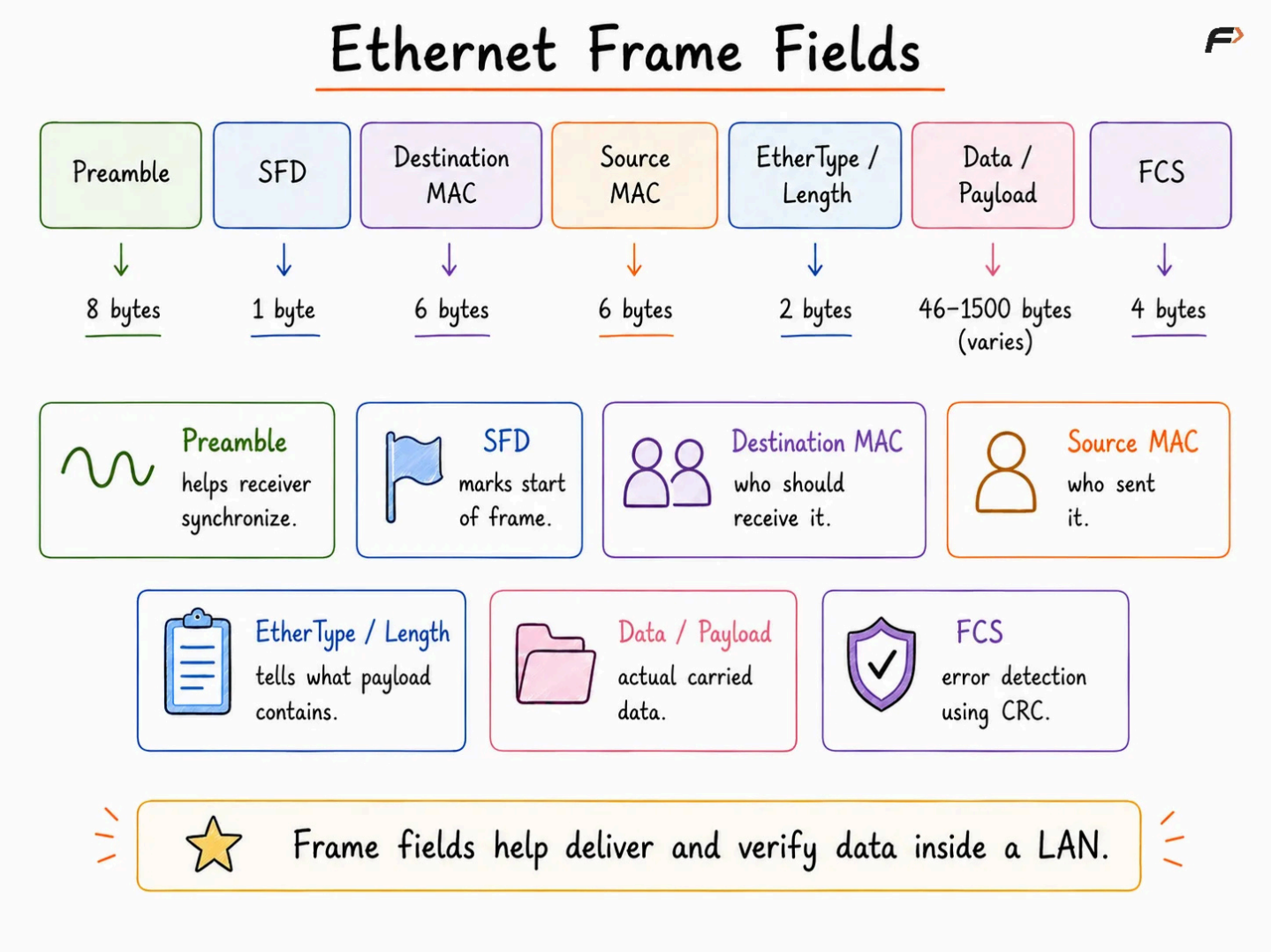
Data (IP Packet)

Source MAC Address

Destination MAC Address

Frame Check Sequence (FCS)

Components of an Ethernet Frame



Preamble (8 Bytes): Used to synchronize the sender and receiver signals before transmission begins.

Start Frame Delimiter (SFD) (1 Byte): Marks the start of the frame.

Destination MAC Address (6 Bytes): Specifies which device on the current network should receive the frame.

Source MAC Address (6 Bytes): Identifies the sender.

EtherType / Length (2 Bytes): Specifies the protocol being carried or the payload length.

Data / Payload (46–1500 Bytes): Contains the IP packet or upper-layer data.

Frame Check Sequence (FCS) (4 Bytes): Used for error detection.

CRC and FCS

CRC (Cyclic Redundancy Check) is an error detection algorithm. Its result is stored inside the Frame Check Sequence (FCS) field.

The receiver recomputes the CRC value and compares it with the received FCS value.

If both values match, the frame is assumed to be valid.

If they differ, the frame is considered corrupted and is discarded.

Padding in Ethernet Frames

Ethernet requires a minimum frame size of 64 bytes. Therefore, the payload portion must contain at least 46 bytes.

If the actual data is smaller than 46 bytes, extra bits called **padding** are added.

Padding does not contain useful information and is ignored by the receiver. Its purpose is to maintain the minimum frame size and ensure reliable communication.

Error Detection: Parity, Checksum and CRC

Whenever data travels through a communication medium, there is always a possibility that some bits may get corrupted. Noise, electrical disturbances, attenuation, interference, or other transmission problems can alter the transmitted bits before they reach the receiver.

For example, suppose the sender transmits a sequence of bits. Due to noise during transmission, one or more bits may change, causing the receiver to receive different data. If the receiver blindly trusts the received bits, incorrect information may be processed.

Therefore, the receiver needs some mechanism to determine whether the received data is valid or whether it has been corrupted during transmission. This process is known as **error detection**.

It is important to understand that error detection does not necessarily mean error correction. Detecting an error simply means identifying that something went wrong.

What happens after an error is detected depends upon the application.

For example, during a live video stream, losing one frame is generally acceptable. The corrupted frame may simply be discarded because waiting for retransmission could interrupt the video.

On the other hand, in applications like email or file transfer, losing even a small amount of data is unacceptable. In such situations, the corrupted data must be retransmitted.

Some common error detection techniques are:

Parity

Checksum

CRC (Cyclic Redundancy Check)

Among these, parity is the simplest technique, checksum is stronger than parity, and CRC is the most powerful and reliable among the three.

Parity

Parity is one of the simplest error detection mechanisms. The basic idea behind parity is to add one extra bit, called the **parity bit**, so that the total number of 1s in the transmitted data follows a particular rule.

Depending on the agreed rule, parity can be of two types:

Even Parity → Total number of 1s should be even.

Odd Parity → Total number of 1s should be odd.

Before communication begins, the sender and receiver agree upon which type of parity will be used. This agreement becomes part of the communication protocol.

Let us understand even parity with examples.

Example 1

Suppose the sender wants to send:

1011

The number of 1s is 3, which is odd.

Since even parity is being used, one extra bit with value 1 is added.

The transmitted data becomes:

10111

Now the total number of 1s becomes 4, which satisfies the even parity rule.

Example 2

Suppose the sender wants to send:

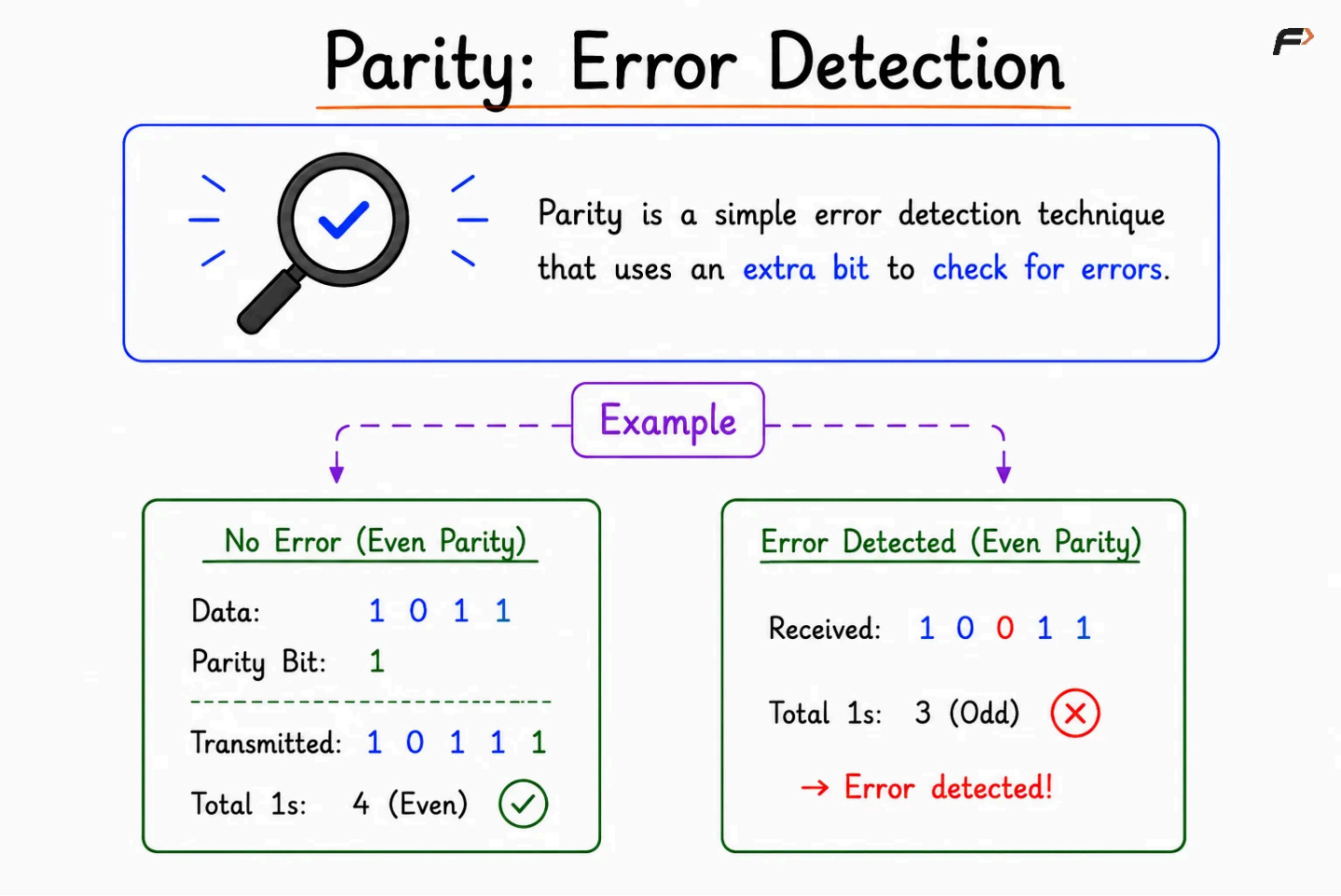
1010

Here the number of 1s is already 2, which is even.

Therefore, no additional 1 is needed. The parity bit is 0.

Hence the transmitted sequence becomes:

10100



When the receiver receives the data, it counts the total number of 1s again.

If the number of 1s follows the agreed parity rule, the receiver assumes that no error has occurred.

If the count violates the parity rule, the receiver concludes that the data has been corrupted.

Single Bit Error Detection

Suppose one bit gets corrupted during transmission.

Because one bit changes, the total number of 1s changes from even to odd (or vice versa). As a result, the receiver can immediately detect that an error has occurred.

Limitation of Parity

Parity works well when a single bit gets corrupted, but it is not perfect.

Suppose two bits are corrupted simultaneously. In that case, the total number of 1s may still remain even. Therefore, the receiver may incorrectly assume that the data is correct.

Thus, parity is a very simple and weak error detection mechanism and cannot reliably detect multiple-bit errors.

Checksum

Checksum is another error detection technique which is stronger than parity.

Instead of simply counting the number of 1s, checksum computes a small numerical value from the entire data and sends that value along with the original data.

At the receiver side, the same computation is performed again.

If both checksum values match, the data is assumed to be correct.

If they do not match, an error is detected.

The working of checksum can be understood in the following steps:

Break the data into smaller units.

Add the units using a predefined rule.

Generate a final value called the checksum.

Send the checksum along with the original data.

The receiver computes the checksum again.

The computed checksum is compared with the received checksum.

Example

Suppose the data is:

01010011

Assume that we divide it into two 4-bit blocks:

0101 and 0011

Their decimal values are:

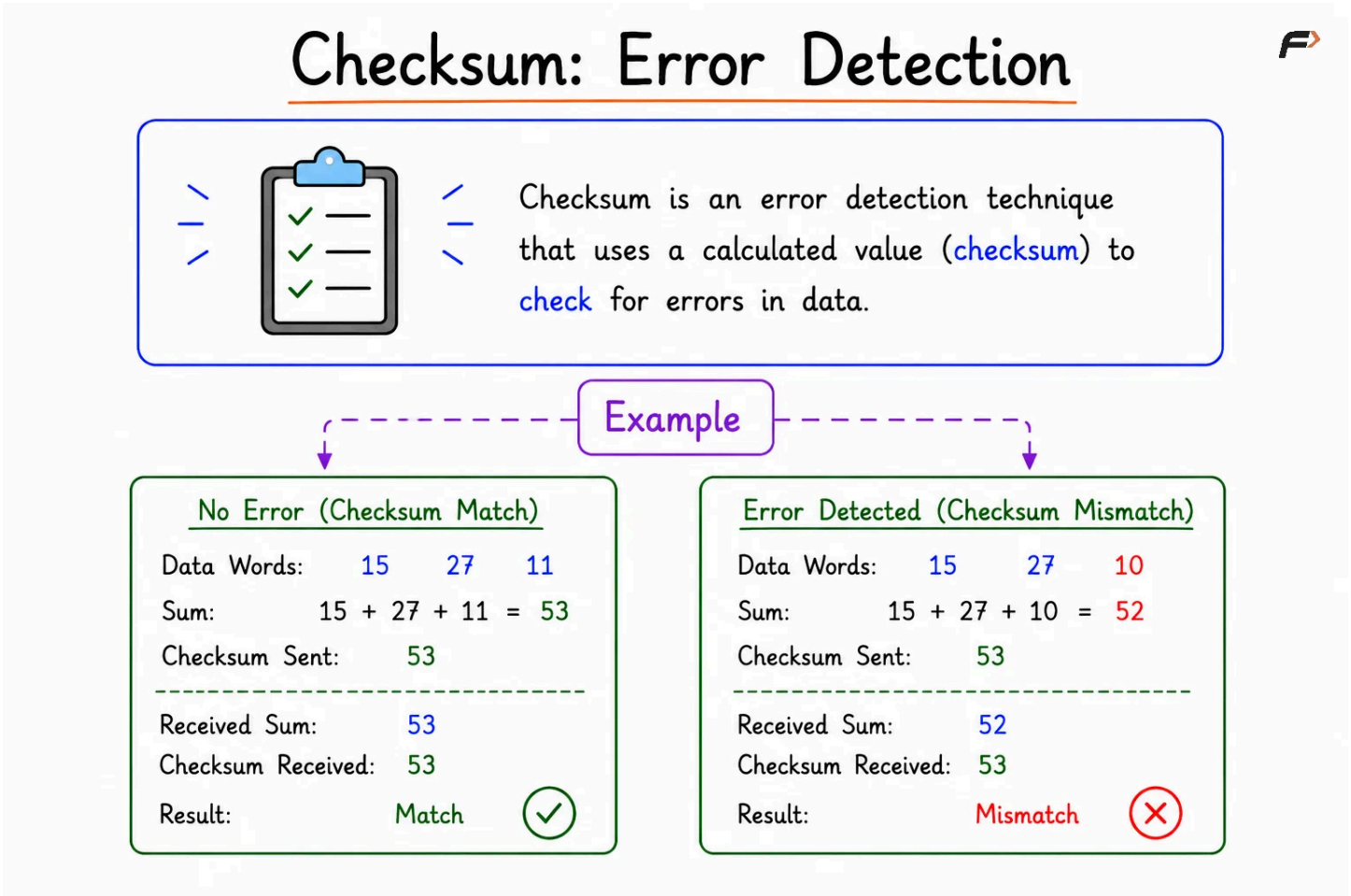
0101 = 5
0011 = 3

Adding them gives:

5 + 3 = 8

Therefore, the checksum becomes:

1000



The receiver performs the same operation again and compares the computed checksum with the received checksum.

If both are identical, the receiver assumes that the data has not been corrupted.

Checksum is stronger than parity because it uses the entire data instead of simply counting the number of 1s.

However, checksum is not perfect. Different corrupted data patterns may sometimes produce the same checksum value. In such cases, errors may remain undetected.

CRC (Cyclic Redundancy Check)

CRC is a much stronger error detection mechanism and is widely used in communication networks.

In CRC, the sender computes a special value from the data using a predefined mathematical rule. This value is transmitted along with the original data.

Both the sender and receiver use the same agreed algorithm.

The overall process is as follows:

Sender takes the data.

Sender applies the agreed CRC algorithm.

A CRC value is generated.

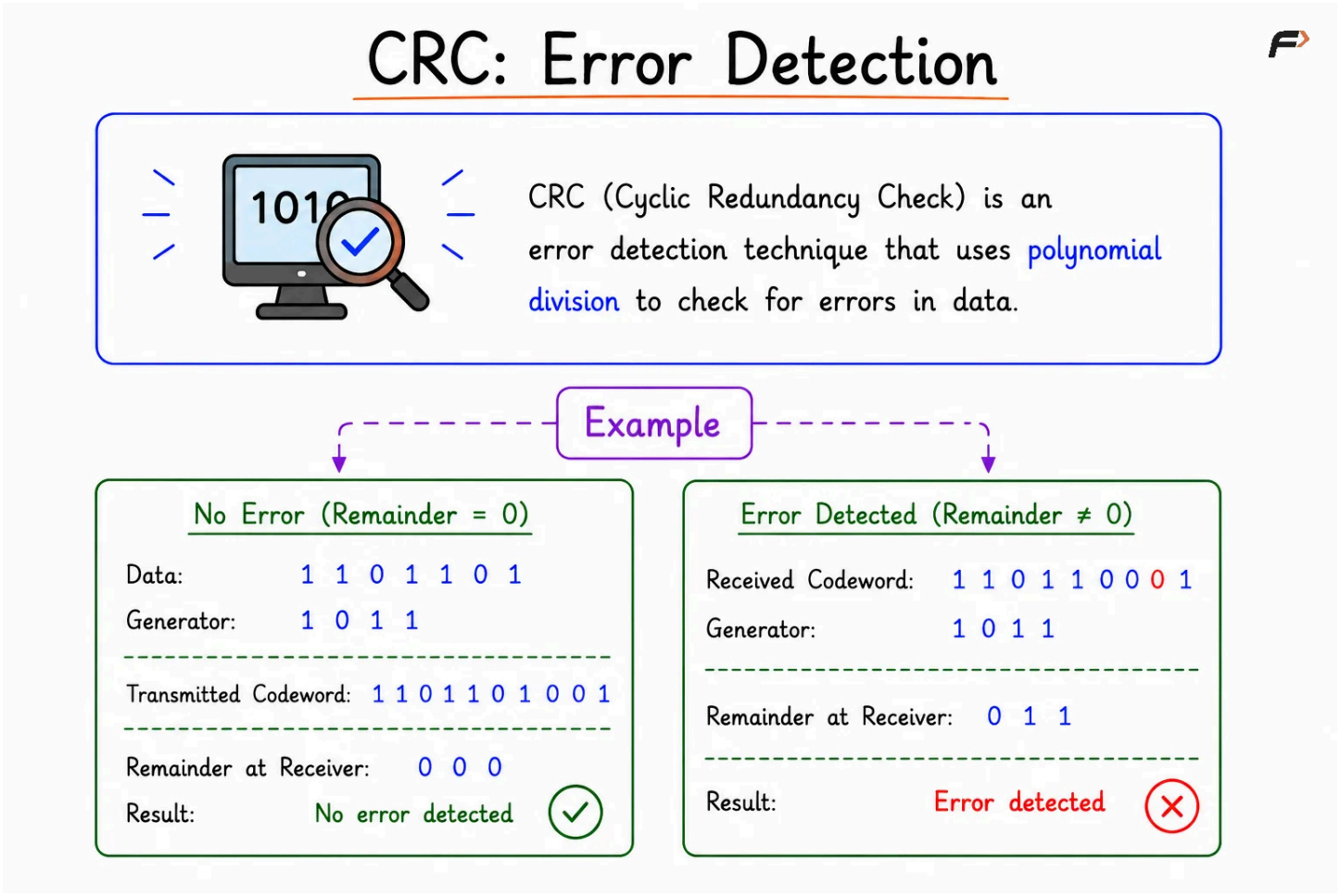
The original data and CRC value are transmitted together.

The receiver receives both.

The receiver performs the same CRC computation again.

If the result is valid, the frame is accepted.

Otherwise, an error is detected.



Internally, CRC uses a predefined mathematical rule agreed upon by the sender and receiver. Because of its stronger error detection capability, CRC is much better at detecting burst errors compared to parity and checksum.

However, just like parity and checksum, CRC only detects errors. It does not automatically correct them.

Quick Comparison

Method	How It Works	Strength	Main Limitation
Parity	Adds one parity bit	Simple	Cannot detect some multi-bit errors
Checksum	Computes a numerical summary of data	Moderate	Different errors may produce the same checksum
CRC	Uses a predefined mathematical rule	Strong	Provides detection but not correction

CSMA/CD vs CSMA/CA

Both CSMA/CD and CSMA/CA are techniques used to solve the shared-medium problem. When multiple devices share the same communication medium, they may attempt to transmit at the same time. Their signals interfere with one another, causing collisions.

Although both mechanisms deal with collisions, they approach the problem differently.

Understanding Collision

Suppose two devices, Device A and Device B, share the same communication medium.

Both devices first check whether the medium is free. Since the medium appears idle to both devices, they decide to transmit at exactly the same time.

As both signals enter the medium simultaneously, they interfere with each other and a collision occurs.

Thus, a collision occurs when multiple devices transmit at the same time and their signals overlap.

Collisions were common in older shared Ethernet environments, especially with hubs and half-duplex communication.

Collisions can lead to:

Corrupted data.

Repeated retransmissions.

Devices entering retry loops.

Reduced network efficiency.

If collisions happen repeatedly, devices spend more time retransmitting than actually sending useful data.

CSMA/CD

CSMA/CD stands for **Carrier Sense Multiple Access with Collision Detection**.

It was primarily used in classical shared Ethernet, hubs, and half-duplex communication.

Carrier Sense

Before transmitting, a device first listens to the medium to determine whether it is currently free.

If the medium is busy, the device waits. If the medium is idle, transmission begins.

Multiple Access

Multiple devices share the same communication medium and can attempt to transmit over that shared channel.

Collision Detection

Even after checking the medium, another device may also decide to transmit at the same time.

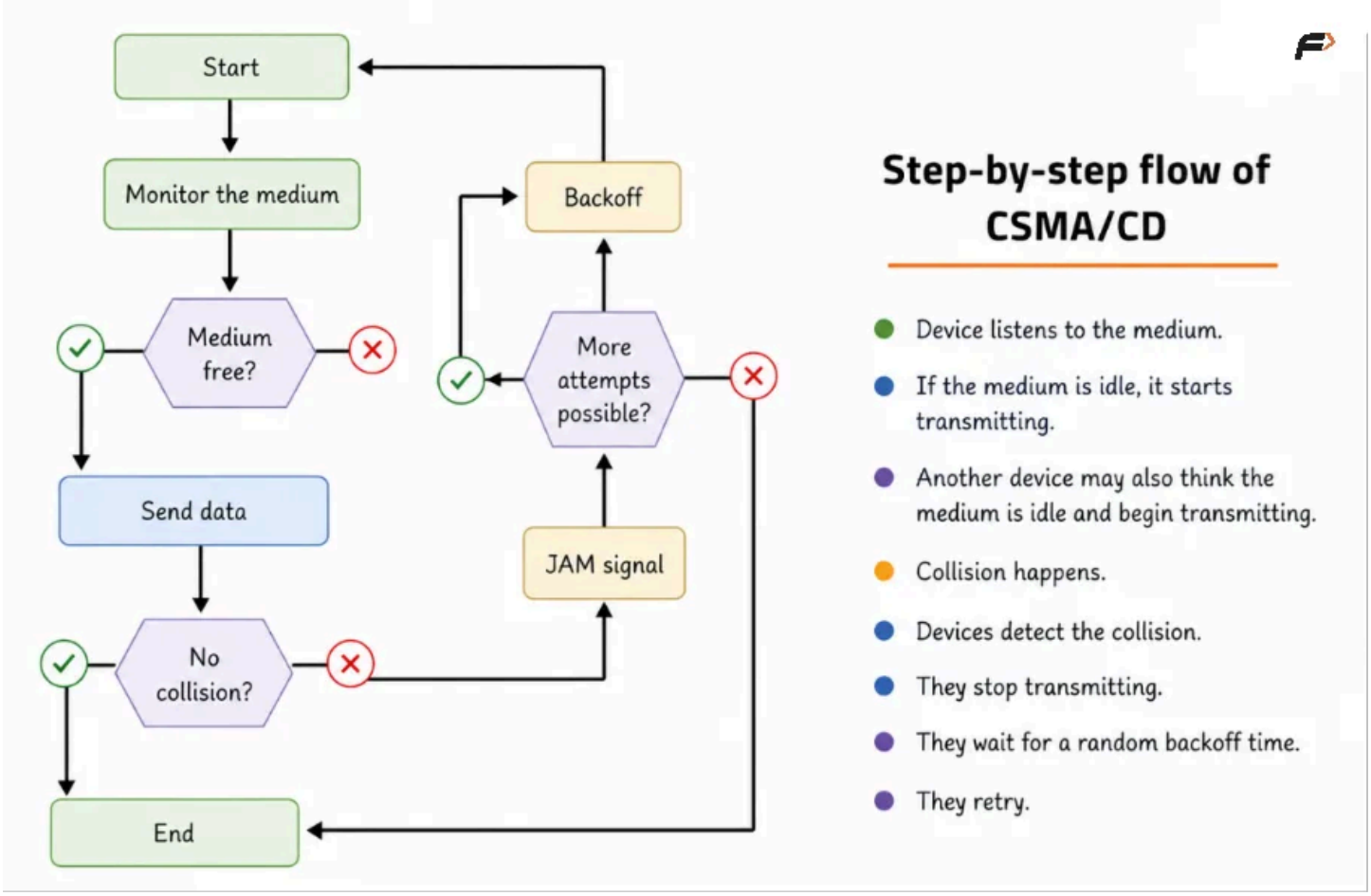
If a collision occurs, the transmitting devices detect the collision while sending data.

Once a collision is detected:

The devices immediately stop transmitting.

They wait for a random backoff time.

They retry transmission later.



Why Random Backoff is Necessary

Suppose Device A and Device B collide.

If both devices wait for exactly the same amount of time and retry together, they may collide again. This process could repeat continuously.

Therefore, every device chooses a random backoff period.

For example:

Device A may wait for 10 seconds.

Device B may wait for 15 seconds.

Since their waiting times are different, the probability of another collision becomes much smaller.

Thus, random backoff helps reduce repeated collisions.

Summary of CSMA/CD

CSMA/CD follows the philosophy:

Transmit → Detect Collision → Stop → Wait → Retry

In other words, collisions are allowed to happen first and are detected afterwards.

CSMA/CA

CSMA/CA stands for **Carrier Sense Multiple Access with Collision Avoidance**.

It is primarily used in Wi-Fi networks.

Unlike CSMA/CD, the objective here is not to detect collisions after they happen, but to avoid them before transmission.

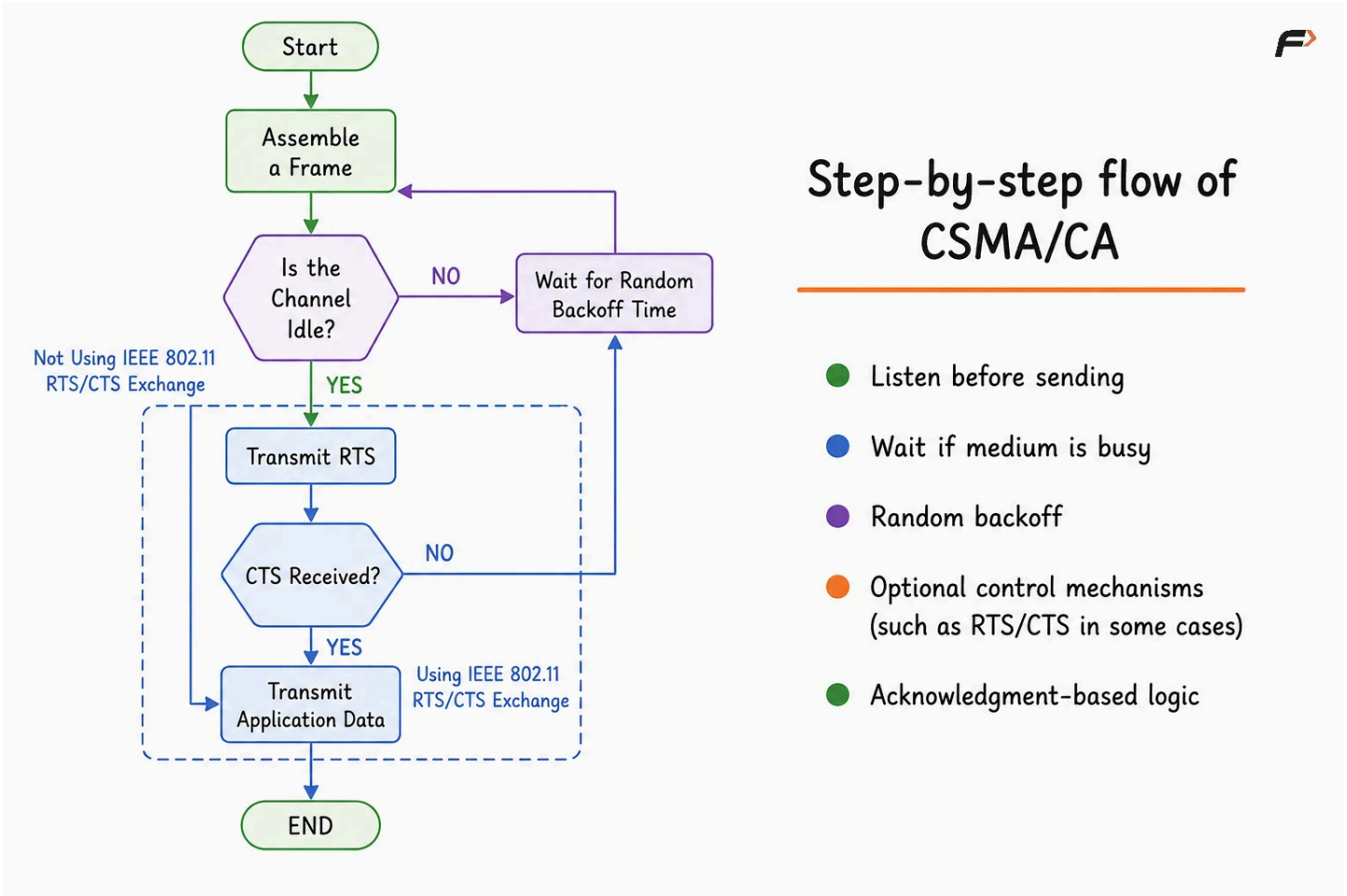
How Collision Avoidance Works

A device first listens to the medium.

If the medium is busy, the device waits.

When the medium becomes free, the device still does not transmit immediately.

Instead, it waits for a random backoff period.



For example:

One device may wait for 5 units of time.

Another may wait for 10 units.

A third device may wait for 15 units.

During this waiting period, if another device starts transmitting, the device postpones transmission and waits again.

Only when the channel remains free throughout the backoff period does the device transmit its data.

This process helps reduce the probability of collisions.

Acknowledgement Mechanism

After transmitting data, the sender waits for an acknowledgement from the receiver.

If an acknowledgement is received, the transmission is considered successful.

If no acknowledgement arrives, the sender assumes that the transmission failed and retries after waiting again.

Therefore, acknowledgements play an important role in CSMA/CA.

Why Wi-Fi Uses Collision Avoidance

Wireless devices generate strong signals while transmitting. Because of this, they cannot reliably detect collisions during transmission in the same way classical Ethernet could.

Therefore, Wi-Fi tries to avoid collisions before they occur instead of detecting them afterwards.

Summary of CSMA/CA

CSMA/CA follows the philosophy:

Listen → Wait → Avoid Collision → Transmit → Receive ACK

Thus, CSMA/CA tries hard to prevent collisions before transmission.

CSMA/CD vs CSMA/CA

CSMA/CD	CSMA/CA
Used in classical Ethernet	Used in Wi-Fi networks
Collision Detection	Collision Avoidance
Collision is detected after transmission	Collision is avoided before transmission
Stop and retry after collision	Wait and transmit carefully
Common in hubs and half-duplex Ethernet	Common in wireless networks

The key difference is:

CSMA/CD: Send first, detect collision later.

CSMA/CA: Try to avoid collision before sending.

Address Resolution Protocol (ARP)

If a machine knows an IP address on the same local network, how does it get the corresponding MAC address so it can build an Ethernet frame? This is exactly what ARP solves.

ARP is the protocol used in IPv4 networks to resolve a known IP address into a MAC address on the local network.

Since Ethernet communication needs a destination MAC address, a host cannot send a frame on a LAN unless it first knows where to send that frame at Layer 2. ARP provides that missing mapping.

Same-LAN ARP Flow

Suppose Laptop A has the IP address **192.168.1.10** and Laptop B has the IP address **192.168.1.20**.

Laptop A checks whether B is in the same subnet → Yes.

A needs B's MAC address, so A checks its ARP cache first.

If an entry exists, it uses that MAC directly.

If not, A sends an ARP Request as a broadcast.

The request is essentially: “Who has 192.168.1.20? Tell me your MAC address.”

What Happens Next?

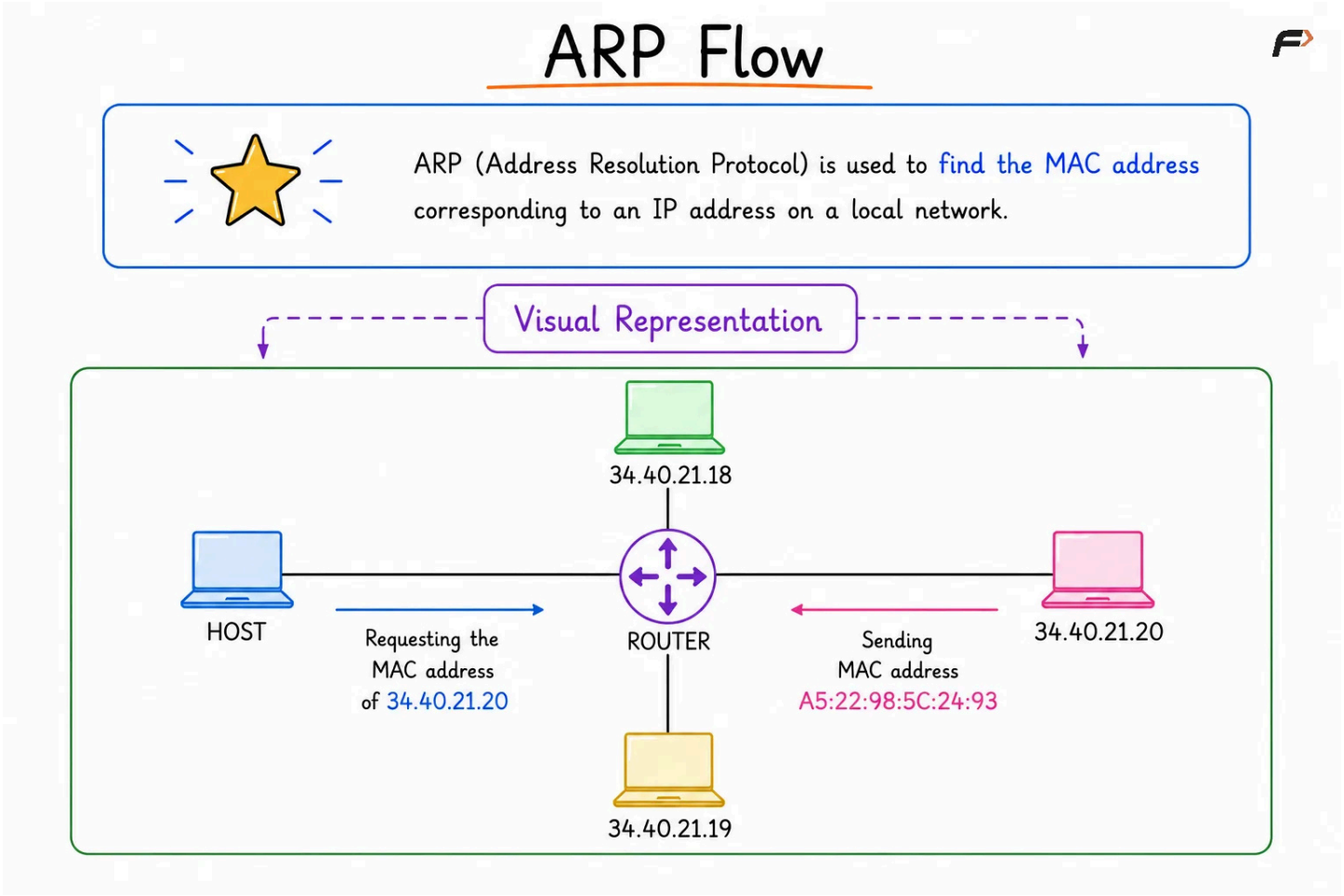
A broadcast destination MAC is used (FF:FF:FF:FF:FF:FF), so all devices on the LAN receive the request. Every device looks at it.

Only the device with IP 192.168.1.20 responds with its MAC address. This is the ARP Reply.

A stores this IP-to-MAC mapping in its ARP cache.

Now A can build the Ethernet frame and send the IP packet.

This is important: IP identifies the destination logically, but the Ethernet frame still needs a physical destination MAC for delivery on the local link.



Different-LAN ARP Flow

If Laptop A wants to reach 8.8.8.8, it does **not** ARP for Google’s MAC address. Instead:

It sees the destination is not in the same subnet.

It decides to send the packet to the default gateway.

It performs ARP for the gateway's MAC address.

The Ethernet frame destination MAC becomes the gateway MAC.

The IP destination still remains 8.8.8.8.

This distinction is critical:

MAC destination = next hop

IP destination = final destination

At every hop along the path, each device:

Looks at the destination IP

Checks its routing table

Decides the next-hop IP

Then finds the next-hop MAC address (often using ARP)

The router rewrites the Layer 2 frame at every hop, while the Layer 3 IP destination generally remains unchanged. This is why MAC addresses can change at each hop, while the destination IP remains the same.

A routing table is built from network configuration, connected routes, static routes, or routing protocols. It helps determine where packets go next.

ARP Cache

An ARP cache is a local table on a host that stores recently learned IP-to-MAC mappings. This avoids repeating ARP broadcasts every time a device sends traffic. Entries typically remain for some time before aging out.

Command to check it: arp -a

Typical ARP cache entry contains:

IP Address

MAC Address

Entry Type (dynamic or static)

Gratuitous ARP

Gratuitous ARP is an ARP announcement sent by a device for its own IP address, usually without being asked.

It is commonly used for:

Announcing “this IP belongs to me”

Updating other devices’ ARP caches

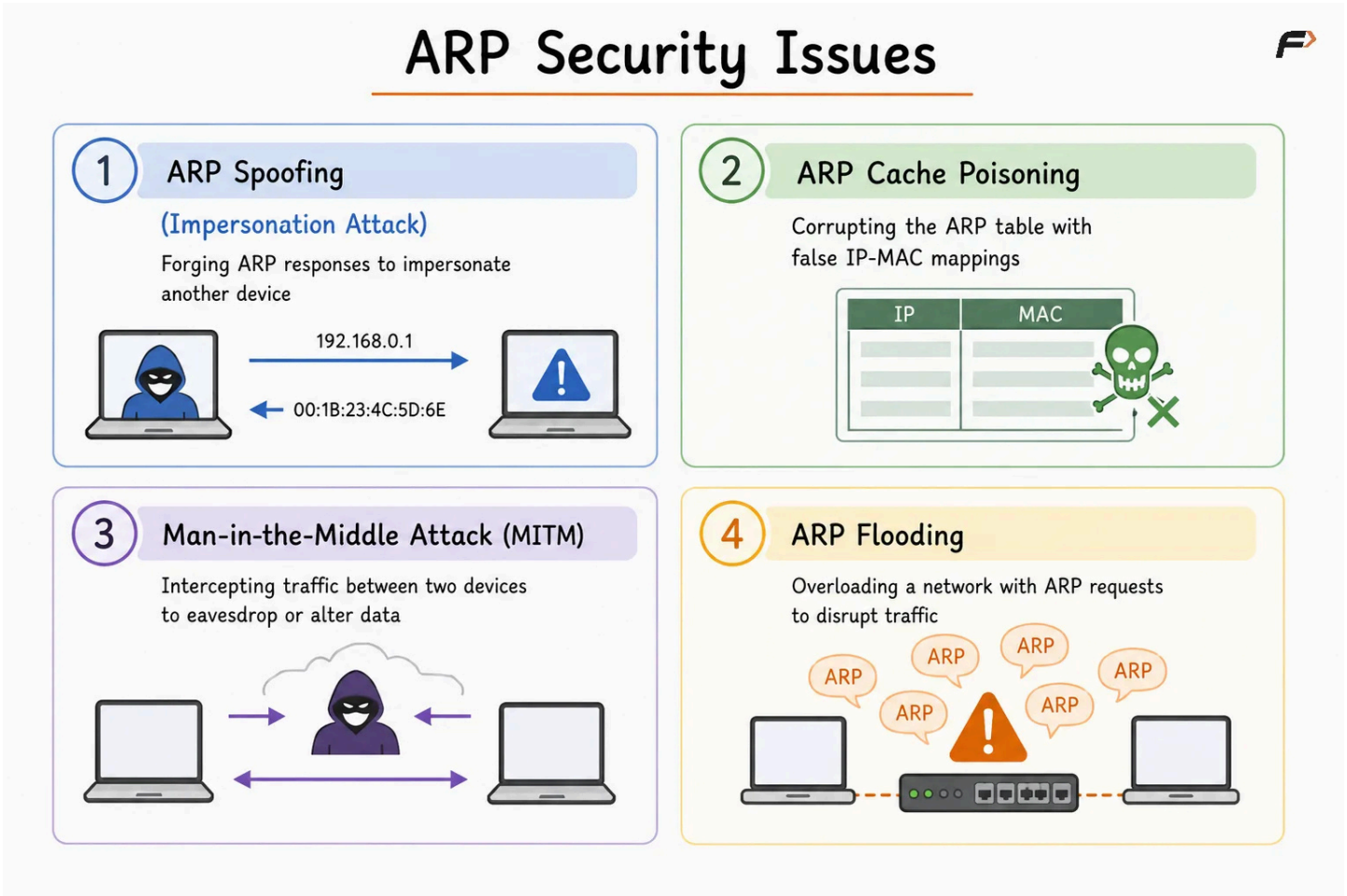
Detecting duplicate IP addresses

Failover scenarios (for example, active-backup systems)

Unlike normal ARP, it is not primarily asking for another device’s MAC. It is an announcement.

ARP Security Issues

ARP does not verify the authenticity of responses, meaning any device can send ARP replies without validation, making it susceptible to attacks. Because ARP was designed for trust-based local communication, it has no built-in authentication mechanism.



ARP Spoofing (Impersonation): An attacker sends forged ARP replies claiming that their MAC address belongs to another device's IP address, such as a gateway. This causes traffic intended for the legitimate device to be redirected to the attacker.

Cache Poisoning: An attacker injects false IP-MAC mappings into a device's ARP cache, causing incorrect routing, data interception, or communication failure. Once poisoned, devices may continue trusting the false mapping until the cache entry expires or is corrected.

Man-in-the-Middle (MITM): The attacker positions themselves between two communicating devices, allowing them to intercept, monitor, or modify data without detection. This often happens as a consequence of successful ARP spoofing.

ARP Flooding: An attacker sends a large number of fake ARP requests or replies to overwhelm switches, potentially exhausting MAC tables and causing switches to behave like hubs, broadcasting traffic more widely.

ARP Security Issues (Prevention)

Dynamic ARP Inspection (DAI): Validates ARP packets by comparing them against a trusted database, blocking unauthorized or fake ARP messages. Typically, switches use DHCP snooping information as the trusted reference.

Static ARP Entries: IP-to-MAC mappings are manually configured and cannot be altered dynamically, preventing attackers from modifying entries using spoofed ARP replies. This is strong but hard to scale in large networks.

Network Segmentation: Dividing the network into smaller VLANs limits broadcast domains, reducing the scope and impact of ARP-based attacks. An attacker in one segment cannot easily affect another.

Port Security: Switch port security limits the number of MAC addresses allowed on a port and can block unknown or suspicious devices. This reduces opportunities for rogue devices to launch attacks.

Network Access Control (NAC)

Network Access Control (NAC) is a security framework that regulates which users and devices can access a network by verifying identity, authentication credentials, and device compliance before allowing communication.

It acts as a gatekeeper for the network. Rather than allowing every connecting device immediate access, NAC checks whether the device is trusted, whether the user is authorized, and whether the endpoint meets security requirements.

This is especially important in enterprise networks where laptops, phones, guest devices, and unmanaged systems may all attempt to connect.

Purpose

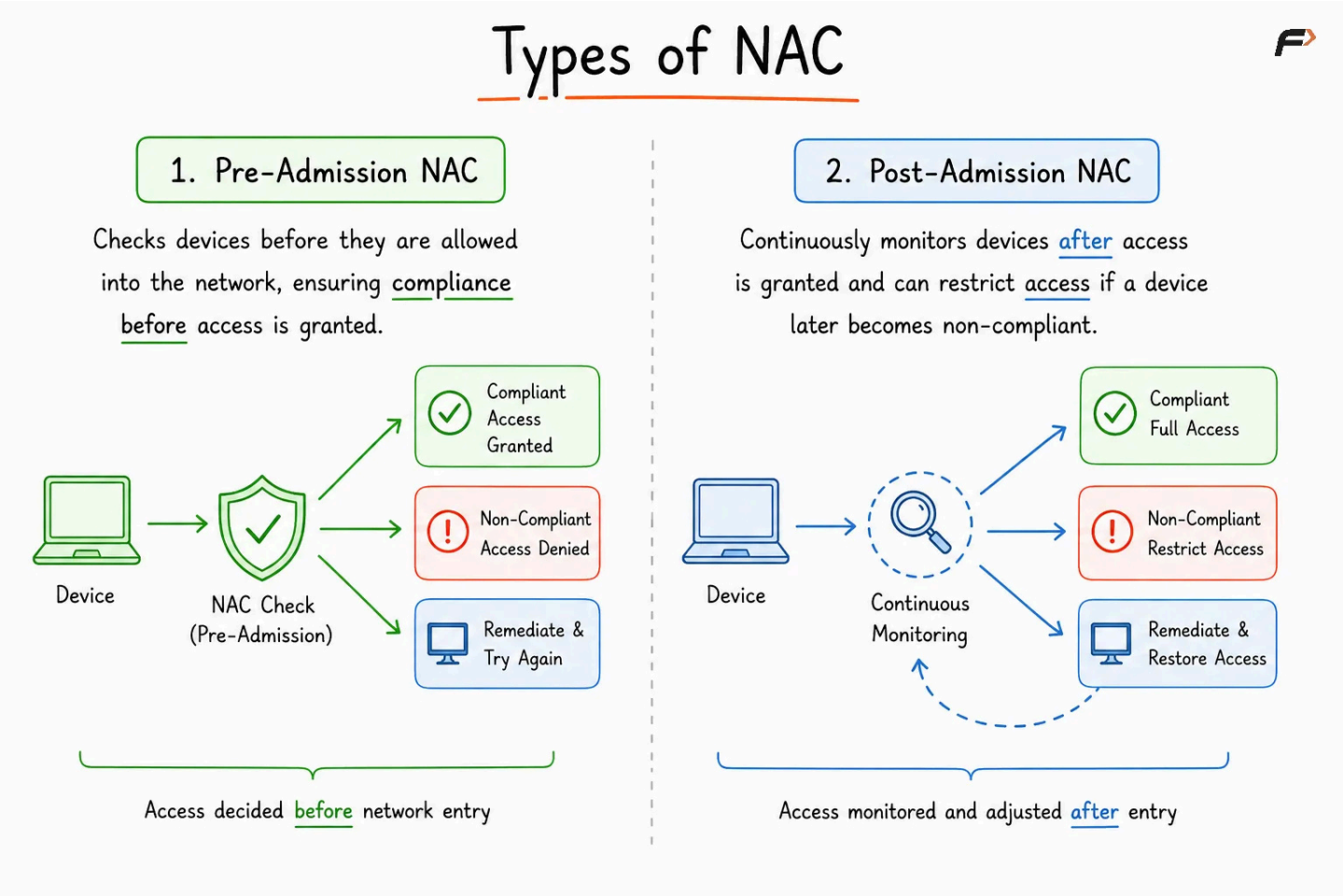
Ensures only authorized and trusted devices are allowed onto the network.

Checks the security posture of devices, such as antivirus status, OS updates, patch levels, and compliance issues.

Enforces security policies dynamically, allowing different levels of access based on user roles, device type, or security status.

Protects the network from compromised or non-compliant devices.

Types of NAC



Pre-Admission NAC

Checks devices before they are allowed into the network, ensuring compliance before access is granted.

Typical checks may include:

User authentication

Device identity verification

Operating system status

Antivirus or endpoint protection checks

Patch and update validation

If requirements are not met, access may be denied or restricted.

Post-Admission NAC

Continuously monitors devices after access is granted and can restrict access if a device later becomes non-compliant.

This matters because devices can become risky after joining the network. For example:

Antivirus may be disabled later.

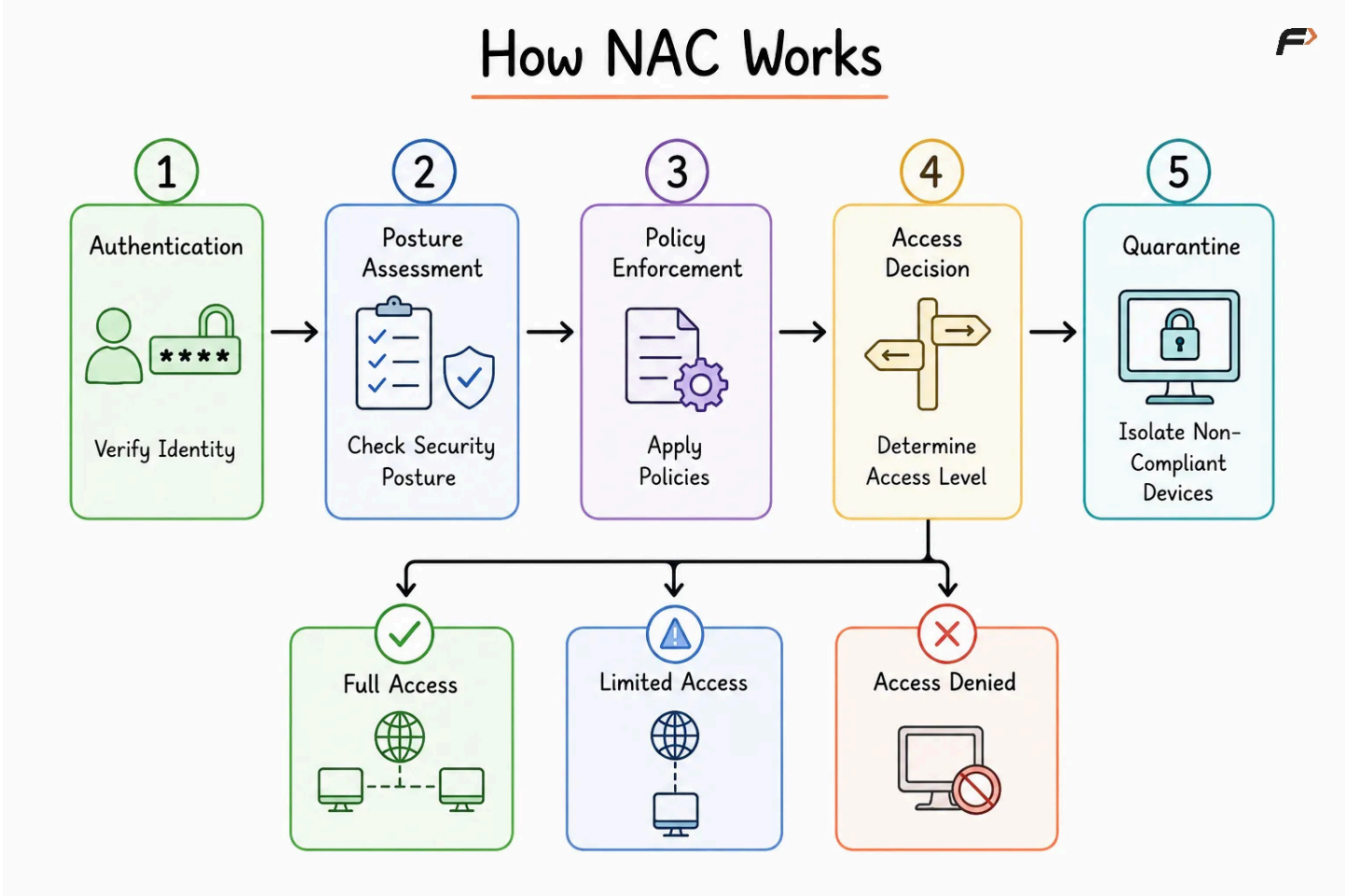
A device may miss required updates.

Malicious behavior may be detected.

Post-admission NAC can react to those changes.

How NAC Works

NAC operates through a multi-step process to validate both the user and the device before granting network access.



Authentication: When a device attempts to connect, NAC verifies the identity of the user or device using credentials such as username/password, certificates, or multi-factor authentication.

Posture Assessment: The system evaluates the device's security condition, checking for factors such as updated antivirus software, installed security patches, and compliance with organizational policies.

Policy Enforcement: Based on authentication and posture results, NAC applies predefined policies to determine the level of access.

Access Decision: The device may be granted full access, limited access, or denied access depending on compliance status.

Quarantine: Non-compliant devices may be placed in a restricted network where they can update or fix issues before gaining full access.

Key Components

Authentication Server: Verifies user or device identity. Often integrated with systems like RADIUS or directory services.

Policy Server: Stores and applies access rules. Determines what level of access should be granted.

Enforcement Points: The devices that actually apply the decision. Examples include switches, routers, firewalls, or wireless controllers.

Advantages of NAC

Enhances network security by preventing unauthorized and insecure devices from connecting.

Provides fine-grained access control based on user roles and device conditions.

Helps organizations maintain compliance with security standards and policies.

Reduces the risk of internal and external security breaches.

Disadvantages of NAC

Requires complex setup and configuration.

Can introduce delays during authentication and security checks.

Requires continuous monitoring, updates, and maintenance.